

Smart Helmet

(IoT based Smart Helmet for better safety & enhanced connectivity)

Capstone Project Report

MID SEMESTER EVALUATION

Submitted by:

(102117148) Lovish Bansal

(102117149) Garv Bundel

(102117157) Soham Kumar

(102117184) Aryan Tyagi

BE 4th Year, COPC

CPG No: 49

Under the Mentorship of

Dr. Vivek Sethi

Assistant Professor



**Computer Science and Engineering Department
Thapar Institute of Engineering and Technology, Patiala
July 2024**

ABSTRACT

The IoT-based Smart Helmet represents a significant leap forward in enhancing motorcycle rider safety and connectivity, leveraging the power of Internet of Things (IoT) technologies. This innovative helmet integrates an array of sensors, including accelerometers and gyroscopes, alongside GPS (from smartphone) and communication modules, to continuously monitor the rider's condition and environment in real time. In the event of a crash, the helmet's advanced sensor suite automatically detects the impact and immediately triggers an emergency response system. This system transmits the rider's exact GPS coordinates (from smartphone) and vital details to the nearest medical facilities, thereby facilitating prompt medical assistance.

The Smart Helmet is not only a tool for immediate accident response but also serves as a proactive measure to encourage responsible riding. By integrating these multifaceted features into a single device, this project aims to significantly reduce road fatalities and redefine safety standards in motorcycle riding. The successful implementation of this project could serve as a model for future innovations in rider safety technology.

DECLARATION

We hereby declare that the design principles and working prototype model of the project entitled Smart Helmet is an authentic record of our own work carried out in the Computer Science and Engineering Department, TIET, Patiala, under the guidance of Dr. Vivek Sethi during 7th semester (2024).

Date:

Roll No.	Name	Signature
102117148	LOVISH BANSAL	
102117149	GARV BUNDEL	
102117157	SOHAM KUMAR	
102117184	ARYAN TYAGI	

Counter Signed By:

Faculty Mentor:

ACKNOWLEDGEMENT

We would like to express our thanks to our mentor Dr. Vivek Sethi. He has been of great help in our venture and an indispensable resource of technical knowledge. He is truly an amazing mentor to have.

We are also thankful to Dr. Shalini Batra, Head, Computer Science and Engineering Department, the entire faculty and staff of the Computer Science and Engineering Department, and also our friends who devoted their valuable time and helped us in all possible ways towards successful completion of this project. We thank all those who have contributed either directly or indirectly towards this project.

Lastly, we would also like to thank our families for their unyielding love and encouragement.

They always wanted the best for us and we admire their determination and sacrifice.

Date: 19th December 2024

Roll No.	Name	Signature
102117148	LOVISH BANSAL	
102117149	GARV BUNDEL	
102117157	SOHAM KUMAR	
102117184	ARYAN TYAGI	

TABLE OF CONTENTS

ABSTRACT.....	i
DECLARATION.....	ii
ACKNOWLEDGEMENT.....	iii
LIST OF FIGURES.....	iv
LIST OF TABLES.....	v
LIST OF ABBREVIATIONS.....	vi
CH APTER.....	Page No.
1. Introduction	
1.1 Project Overview	12
1.2 Need Analysis	14
1.3 Research Gaps	15
1.4 Problem Definition and Scope	17
1.5 Assumptions and Constraints	18
1.6 Standards	19
1.7 Approved Objectives	20
1.8 Methodology	20
1.9 Project Outcomes and Deliverables	22
1.10 Novelty of Work	23
2. Requirement Analysis	
2.1 Literature Survey	25
2.1.1 Theory Associated With Problem Area	25
2.1.2 Existing Systems and Solutions	25
2.1.3 Research Findings for Existing Literature	26
2.1.4 Problem Identified	28
2.1.5 Survey of Tools and Technologies Used	29
2.1.6 Summary	29
2.2 Software Requirement Specification	30
2.2.1 Introduction	30
2.2.1.1 Purpose	30
2.2.1.2 Intended Audience and Reading Suggestions	31
2.2.1.3 Project Scope	32
2.2.2 Overall Description	32
2.2.2.1 Product Perspective	32
2.2.2.2 Product Features	33
2.2.3 External Interface Requirements	34
2.2.3.1 User Interfaces	34

2.2.3.2 Hardware Interfaces	35
2.2.3.3 Software Interfaces	36
2.2.4 Other Non-functional Requirements	38
2.2.4.1 Performance Requirements	38
2.2.4.2 Safety Requirements	38
2.2.4.3 Security Requirements	39
2.3 Cost Analysis	39
2.4 Risk Analysis	40
3. Methodology Adopted	
3.1 Investigative Techniques	42
3.2 Proposed Solution	44
3.3 Work Breakdown Structure	46
3.4 Tools and Technology	49
4. Design Specifications	
4.1 System Architecture	50
4.2 Design Level Diagrams	52
4.3 User Interface Diagrams	57
4.4 Snapshots of Working Prototype	60
5. Implementation and Experimental Result	
5.1 Experimental Setup	61
5.2 Experimental Analysis	63
5.2.1 Data	63
5.2.2 Performance Parameters	63
5.3 Working of the Project	64
5.3.1 Procedural Workflow	64
5.3.2 Algorithmic Approaches	67
5.3.3 Project Deployment	73
5.3.4 System Screenshots	74
5.4 Testing Process	77
5.4.1 Test Plan	77
5.4.2 Features to be tested	78
5.4.3 Test Strategy	80
5.4.4 Test Techniques	80
5.4.5 Test Cases	81
5.4.6 Test Results	84
5.5 Results and Discussions	88
5.6 Inferences Drawn	89
5.7 Validation of Objectives	89
6. CONCLUSIONS AND FUTURE DIRECTIONS	
6.1 Conclusions	92
6.2 Environmental, Economic and Societal Benefits	92
6.3 Reflections	93
6.4 Future Work	94

7.	PROJECT METRICS	
	7.1 Challenges Faced	95
	7.2 Relevant Subjects	96
	7.3 Interdisciplinary Knowledge Sharing	97
	7.4 Peer Assessment Matrix	98
	7.5 Role Playing and Work Schedule	98
	7.6 Student Outcomes Description and Performance Indicators (A-K Mapping)	99
	7.7 Brief Analytical Assessment	101
	 APPENDIX A: References	 103
	APPENDIX B: Plagiarism Report	104

LIST OF FIGURES

Figure No.	Caption	Page No.
1	Block Diagram for Smart Helmet	50
2	Architecture Tier Diagram for Smart Helmet	51
3	DFD (Level 0) for Smart Helmet	52
4	DFD (Level 1) for Smart Helmet	53
5	DFD (Level 2) for Smart Helmet	54
6	Entity Relationship Diagram for Smart Helmet	55
7	Class Diagram for Smart Helmet	56
8	Use Case Diagram for Smart Helmet	57
9	Activity Diagram for Smart Helmet	59
10	Diagram of Prototype	60
11	Procedural Flow	67
12	Component Diagram	74
13	User Signup	74
14	User Login	75
15	User Home page with GPS tracking	75
16	Hospital Signup	76
17	Hospital Login	76

18	Hospital Homepage	77
19	Emailable Report	84
20	Index Report	85
21	XSL Report	87
22	Work Schedule	99

LIST OF TABLES

Table No.	Caption	Page No.
1	Assumptions and Constraints	18
2	Research Findings	26
3	Cost Analysis	39
4	Investigation Techniques	42
5	Login with wrong credentials	82
6	Login with correct credentials	82
7	Sign up functionality	83
8	Logout functionality	83
9	Peer Assessment	98
10	Role played by each team member	98
11	A-K mapping	100

LIST OF ABBREVIATIONS

IoT	Internet of Things
GPS	Global Positioning System
SMS	Short Message System
TIET	Thapar Institute of Engineering and Technology
CSED	Computer Science and Engineering Department
GSM	Global Systems for mobile
GPRS	General Packet Radio Service
IRJET	International Research Journal of Engineering and Technology
IEEE	Internet of Things
GDPR	General Data protection regulation
HTTPS	Hypertext Transfer Protocol Secure
MQTT	Message Queuing Telemetry Transport
ISO	International Organization for Standardization
EIC	Electronics-Instrumentation and Control
MCU	Micro-controller Unit
ID	Identification document
RF	Radio Frequency
HD	High Definition
UI	User Interface

1.1 Project Overview

1.1.1 Technical Terminology

The IoT-based Smart Helmet consists of several key components that work together to provide a comprehensive safety solution. The helmet will be equipped with accelerometers and gyroscopes to detect sudden impacts or abnormal movements that may indicate a crash. These sensors will continuously monitor the rider's condition and trigger an emergency response when necessary. The GPS data is taken from the rider's smartphone's location in real-time, providing accurate data in the event of an accident. Communication modules will be used to send emergency alerts to medical facilities.

A NodeMCU code will be developed to analyse data from the sensors and accurately detect crashes. This code will minimize false crash and ensure that emergency alerts are only triggered in genuine emergencies. A secure, user-friendly dashboard will be developed to allow users or hospitals to monitor the rider's condition and location. The dashboard will be accessible from various devices and provide real-time data and insights. In the event of a crash, the helmet will automatically send an alert containing the rider's GPS coordinates and critical information to nearby medical facilities. This system will ensure that help is dispatched quickly, reducing the time between an accident and medical intervention.

1.1.2 Problem Statement

The increasing number of motorcycle-related accidents and fatalities underscores the need for advanced safety measures that go beyond traditional protective gear. Motorcycles, due to their design, expose riders to a higher risk of injury and death compared to other vehicles. While helmets have been proven to significantly reduce head injuries, they do not provide active safety features such as crash detection, emergency response, or behaviour monitoring.

In many cases, riders who are involved in accidents are left unattended for crucial minutes, sometimes even hours, before help arrives, leading to preventable fatalities. Furthermore, there is a growing need for systems that can monitor and promote safe riding practices, particularly for younger or inexperienced riders.

1.1.3 Goal

The motivation behind the IoT-based Smart Helmet project is to create an intelligent system that enhances rider safety by detecting accidents in real-time, automatically alerting emergency services, and enabling continuous monitoring of riding behaviour. By leveraging IoT technology, this project seeks to provide a proactive and reactive solution that not only reduces response times in emergencies but also encourages responsible riding.

Developing a Smart Helmet prototype includes designing and constructing a helmet equipped with sensors (accelerometers, gyroscopes), and communication modules to enable real-time crash detection and location tracking. To implement an automated emergency response system, we create a system that automatically triggers an emergency response in the event of a crash. GPS coordinates are sent using the rider's smartphone and critical information is transmitted to nearby medical facilities.

To ensure scalability and future enhancements, the system architecture will be designed to accommodate future improvements, such as the integration of additional IoT devices, expanded sensor capabilities, and broader data analytics for deeper insights into rider behaviour.

1.1.4 Solution

The implementation of the IoT-based Smart Helmet will be carried out in several phases. The initial phase will involve designing the helmet's hardware and software components, including the selection of sensors, GPS data, and communication modules. A prototype will be constructed and tested to ensure that all components work together seamlessly.

A custom algorithm will be developed to analyse data from the helmet's sensors and accurately detect crashes. This algorithm will be tested and refined to minimize false positives and ensure reliability. A secure, user-friendly dashboard will be developed to allow parents or guardians to monitor the rider's condition and location. The dashboard will be accessible from various devices and provide real-time data and insights.

Extensive testing will be conducted to validate the functionality and reliability of the Smart Helmet. This will include crash detection accuracy, GPS tracking precision (from smartphone), and user interface usability. The final phase will involve deploying the Smart Helmet to users and providing training on its operation and maintenance. User feedback will be collected to identify any areas for improvement.

The IoT-based Smart Helmet project is expected to deliver a fully functional smart helmet prototype that significantly enhances rider safety through real-time crash detection, automated emergency response, and continuous monitoring of riding conditions. The project will also produce comprehensive technical documentation, a user manual, and a strategic plan for future enhancements.

1.2 Need Analysis

With the world's population growing every year, the number of vehicles on the road also increases. Road accidents are a significant issue for everyone. Every day, a considerable amount of valuable lives are lost as a result of accidents. With the number of two-wheeler deaths only in India increased by nearly 8% in 2022, accounting for 44% of total road fatalities. A solution for biker's safety is immediately needed. It is the most critical field that requires significant exploration, given the high rates of fatalities related to road crashes. At 75,000, record number of bike riders lost lives in 2022 this issue is becoming of more concern.

The Smart Helmet IoT Device is a solution designed to significantly enhance rider safety and connectivity on the road. By integrating advanced IoT technologies and a combination of sensors, we can continuously monitor various parameters and valuable insights for both riders and their families. In the event of a crash, the integrated sensors swiftly trigger an

emergency signal transmission to nearby medical facilities, while simultaneously relaying precise GPS coordinates using rider's smartphone and essential rider information for expedited assistance. A report of the Law Commission of India states that 50% of those who currently die in accidents, could be saved if they receive timely medical attention. It's a very huge percentage and must be reduced because otherwise it will be always uncontrolled and those who can be saved will not be saved. So this solution will surely help in bringing down this percentage.

The user-friendly interface not only promotes transparency and accountability but also serves as a valuable tool for parental control purposes. Also knowing that they are equipped with a helmet that offers advanced safety features and immediate assistance in case of emergencies can provide users and their loved ones with peace of mind, encouraging safer behaviours and promoting overall well-being.

1.3 Research Gaps

1. Anitha, Anusha, Prema, Rumana Anjum, Akshatha , “Smart Helmet for Safety and Accident Detection using IOT,” International Research Journal of Engineering and Technology (IRJET), vol. 06 Issue, p. 4, 2019.

Limitation: The contact may be busy missing the alert SMS, no alert received on device as it may be on the silent mode or ran out of battery.

2. V. R. Ch, S. Chegondi, K. Myla, S. Murala, S. S. Myla and D. R. Kalava , “IoT Based Smart Helmet for Bike Rider Safety,” IEEE, no. 10.1109, p. 4, 2023.

Limitation: The pre-defined contact may be out of reach due to several reasons, alcohol detecting sensor be covered or blocked with some substance.

3. Rahul Kamdi, Rohan Vaidya, Deepali Borakhade, Prasheel Thakre, Ajinkya Nilawar, Vishal Panchbhai , “An IoT Based Intelligent and Smart Helmetfor Bike RidersUsing Arduino,” International Journal of Intelligent Systems and Applications in Engineering, vol. 12, no. 2147-6799, p. 6, 2024.

Limitation: Only single sensor used for the crash detection which is less efficient, alcohol detector could be tempered easily, only a single emergency contact used.

4. M. E. Alim, S. Ahmad, M. N. Dorabati and I. Hassoun , “Design & Implementation of IoT Based Smart Helmet for Road Accident Detection,” IEEE, vol. 11, no. 2644 3163, p. 5, 2020.

Limitation: Undetected sudden jerk or wrong jerk detection on speed breaker or pothole, no emergency signal to nearest hospital for immediate help.

5. Monagi H. Alkinani , Abdulwahab Ali Almazroi, NZ Jhanjhi, and Navid Ali Khan , “5G and IoT Based Reporting and Accident Detection (RAD) System to Deliver First Aid Box Using Unmanned Aerial Vehicle,” MDPI Journals, vol. 21, no. 6905, p. 16, 2021.

Limitation: If smartphone got damaged in the crash the readings for detection will be difficult to collect, no full coverage of 5G at most places and weak signals results in lower performance.

6. Guntupalli Sireesha, Anusha N, K. Baby Satya Jahnavi, Ayusha Baburay , “Smart Helmet using IoT,” International Journal of Engineering Research & Technology (IJERT), vol. 8, no. 14, p. 5, 2020.

Limitation: No alert message to the nearby hospital for immediate help, registered emergency contact may be unreachable.

7. Md. Atiqur Rahman, S.M Ahsanuzzaman, Ishman Rahman, Toufiq Ahmed, Abid Ahsan , “IoT Based Smart Helmet and Accident Identification System,” IEEE Xplore, vol. IEEE Region 10 Symposium, no. 2642-6102, p. 5, 2020.

Limitation: No website support for the database system, reliability only on one sensor for the crash detection, IR sensor may not be reliable in all lighting conditions.

8. Mangesh Jadhawar, Gauri Kandepalli, Ashlesha Kohade, Rajkumar Komati , “SMART HELMET SAFETY SYSTEM USING ATMEGA 32,” International Journal of Research in Engineering and Technology, vol. 05, no. 05, p. 3, 2016.

Limitation: No GPRS for knowing the exact location of the user, bluetooth facility absent which is helpful in better connectivity.

1.4 Problem Definition and Scope

Motorcycle riders face a higher risk of accidents compared to other vehicle users, largely due to the lack of protective barriers and advanced safety systems available in cars. Traditional helmets, while essential for protecting against head injuries, do not address other critical safety concerns such as crash detection, emergency response, and rider behaviour monitoring. In many cases, when accidents occur, the delay in providing timely medical assistance can lead to severe consequences, including increased fatalities. Furthermore, there is a growing need for systems that can monitor and promote safe riding practices, particularly for younger or inexperienced riders. The existing solutions are either too fragmented or lack real-time, automated emergency response capabilities, creating a gap in the market for an integrated, comprehensive safety solution.

The IoT-based Smart Helmet project aims to bridge this gap by developing an intelligent, connected helmet that enhances rider safety through a combination of real-time monitoring, automated emergency response, and behaviour analysis.

This project focuses on creating an embedded system capable of detecting crashes in real time using accelerometers and gyroscopes, with GPS integration using user's smartphone to accurately determine the location of accidents. Upon detecting a crash, the system will automatically send emergency alerts, including the rider's coordinates, to nearby medical services. The project will also ensure that the user interface is intuitive and easily accessible, providing a seamless experience across web platforms. Furthermore, the scope includes laying the groundwork for future scalability, with potential enhancements such as integration with additional IoT devices, expanded sensor capabilities, and broader data analytics, making the solution adaptable for various transportation modes and user needs.

1.5 Assumptions and Constraints

Table 1: Assumptions and Constraints

S.No.	Assumptions
1	It is assumed that the nearest medical facility to the crash location will be the one to send medical assistance, regardless of the fastest route or alternative medical facilities in the area. This assumption simplifies the decision-making process for emergency response. The system prioritizes proximity over route efficiency in emergency situations.
2	The system assumes that there will be a stable internet connection available at all times to transmit data from the helmet to the server and send emergency alerts.
3	It is assumed that the GPS data will be provided by user's smartphone accurately which is crucial for emergency assistance and trip monitoring.
4	The sensors used in the helmet (accelerometer, gyroscope, etc.) are assumed to be reliable and capable of accurately detecting crashes.
5	The user is assumed to wear the helmet properly and charge it regularly to ensure that all systems are functional.
6	The project assumes continuous availability of the server to process data and provide real-time updates on the web platform.
7	It is assumed that the mobile network will have adequate coverage in the areas where the helmet is used to send alerts to emergency contacts.
8	The helmet's battery is assumed to have sufficient capacity to support all functionalities during a typical ride.
9	The project assumes that the helmet will be used in standard weather conditions and that extreme weather (e.g. rain) will not impair the functionality of the sensors and communication modules.

1.6 Standards

Data Privacy and Security Standards:

- **Data Protection Regulations (e.g., GDPR):** The Smart Helmet system must comply with relevant data protection regulations, such as the GDPR, to ensure the privacy and security of user data. This includes safeguarding the rider's personal information, GPS coordinates (from smartphone), and trip data from unauthorized access or breaches.
- **Secure Communication Protocols:** The system should employ secure data transfer protocols, such as HTTPS and MQTT, to ensure that data transmitted between the helmet, user interface, and cloud servers is encrypted and protected against interception or tampering.

Safety and Performance Standards:

- **ISO 26262 (Functional Safety for Road Vehicles):** The design and implementation of the helmet's electronic systems should adhere to ISO 26262, which covers the functional safety of electrical and electronic systems in road vehicles. This ensures that critical safety functions, like crash detection and emergency response, are reliable and fault-tolerant.
- **IEC 61508 (Functional Safety of Electrical/Electronic/Programmable Electronic Safety-related Systems):** The helmet's systems should comply with IEC 61508, a standard that addresses the safety lifecycle requirements for electronic systems, including risk assessments, safety integrity levels, and validation processes.

Software Development Standards:

- **Modular Design and Code Reusability:** The software components of the Smart Helmet should be designed in a modular fashion, allowing for easy updates, maintenance, and scalability. Reusable code components can also streamline the development process and reduce potential errors.
- **Rigorous Code Reviews and Testing:** Regular and rigorous code reviews, along with automated testing (unit, integration, and system testing), should be conducted to identify and fix vulnerabilities, ensuring the software is robust and secure.

- **Continuous Integration and Deployment (CI/CD):** The development process should follow CI/CD practices, enabling continuous integration of code changes and automated deployment of updates. This approach helps in maintaining the reliability and security of the software throughout its lifecycle.

Environmental and Safety Standards:

- **Electromagnetic Compatibility (EMC) Standards:** The helmet's electronic components should comply with EMC standards to prevent interference with other devices and ensure the helmet operates safely and effectively in various environments.
- **Environmental Sustainability:** The design and production of the Smart Helmet should consider environmental sustainability, minimizing the carbon footprint through efficient use of materials and energy, and promoting the use of recyclable components.
- **Durability and Weather Resistance:** The helmet must meet or exceed durability standards, such as those set by ASTM (American Society for Testing and Materials) or similar bodies, ensuring it can withstand various weather conditions and impacts.

1.7 Approved Objectives

- To automatically send precise coordinates and rider details to nearby medical facility upon detecting a crash.
- To trigger a beeping sound from the device upon detecting a crash through the sensors.
- To develop a web-interface that provides rider information to the user and provides crash location to the hospital web-interface along with rider's information.

1.8 Methodology Used

Phase-1: Learning about sensors and identifying the right ones for the accomplishment of all the objectives. Selecting the right hardware including Accelerometer, Gyroscope, Arduino, IoT device (Node MCU), Buzzer, Display, Bread Board, wires, battery, etc. After

the selection of the right hardware, getting them available and move to the process of assembling.

Phase-2: Assembling the hardware and programming the Arduino/NodeMCU with code to carry out all the essential functionalities like:

- Transmitting the alert signal through Node MCU as a jerk of certain intensity is detected through sensors to the server which then later uses this signal with GPS coordinates from the user's smartphone to find the nearest hospital and communicate the rider's coordinates and additional information with an emergency alert for immediate help.

Phase-3: After the successful assembling and uploading of the code to Node MCU, we need to build a database that automatically stores the data received from Node MCU transmitted over the internet on regular time intervals. This data will consist of elements like:

- Coordinates received after every few seconds.
- Time frame of every coordinates received.
- Rider's and/or pre-defined guardian's usernames and passwords.

Phase-4: Next we create a website, overall functionality of this website can be defined by explaining the operations of its two ends, the backend takes the raw data from the database, processes it into useful data and the front end displays it so the end user can interact with it. Each device will have a unique number and these number will be linked with the usernames created on the site so that one person can only interact with his/her own device's data. User will have to give this unique ID of the device while creation of a new account on the website's login page. The backend will be responsible for comparing and finding the nearest medical assistance from the location of detected crash.

Phase-5: This is the last phase of the project. Integration of all elements and smooth working of all sensors, devices, code and software is going to be tested, walking through the whole process again to identify edge case and fix code or hardware accordingly. Lastly, testing of the complete system in different scenario to detect and fix anything if needed.

1.9 Project Outcomes and Deliverables

1. Hardware and Software Components of the Integrated Smart Helmet

The fully integrated Smart Helmet will consist of both hardware and software components designed to enhance motorcycle rider safety and connectivity. The hardware features include:

Sensors and Modules: Integration of accelerometers, gyroscopes, GPS coordinates from the smartphone, and Node MCU for data collection and processing.

Communication Systems: Implementation of communication modules for real-time data transmission between the helmet, external devices, and emergency services.

Display and Alert Systems: A display unit for real-time feedback and alerts, and a buzzer for audio warnings.

Power Management: Efficient power supply systems including batteries and power management circuits.

The software components will include:

User Interface: A web-based interface for crash details, location details of the crash, rider's basic information and accessing trip data.

Emergency Response System: Software for automatic crash detection and alerting nearest emergency services.

2. Crash Detection and Response Algorithms

Crash Detection: Utilizing sensor integration and data to recognize crash events and differentiate them from normal riding conditions.

Real-Time Response: Immediate activation of emergency response protocols, including sending alerts to medical facilities and emergency contacts.

Data Accuracy: Ensuring high accuracy in crash detection and minimal false positives through rigorous testing and algorithm refinement.

3. Integration with Existing Helmet Infrastructure

The Smart Helmet project will involve integrating new technologies with existing helmet infrastructure:

Compatibility: Ensuring that the added components (sensors, modules, etc.) integrate seamlessly with the existing helmet structure without significant alterations.

Flexibility: Designing the system to be adaptable to various helmet types and sizes, ensuring that it can be used across different motorcycle models and rider preferences.

4. Sustainable and Energy-Efficient Design

The project will incorporate sustainable practices and energy-efficient components:

Energy-Efficient Hardware: Use of low-power sensors, efficient communication modules, and optimized power management to reduce overall energy consumption.

Optimization Algorithms: Implementing algorithms that minimize unnecessary processing cycles and energy usage, enhancing the system's efficiency.

Environmental Impact: Reducing the carbon footprint by promoting safer riding practices and minimizing the need for frequent emergency interventions.

1.10 Novelty

Crash Detection and Emergency Response: The helmet's integration of deep learning algorithms and advanced sensor technology allows for precise and rapid detection of crashes. In the event of an accident, the system triggers an automatic emergency response, sending the rider's exact GPS coordinates from user's smartphone and vital information to emergency services. This instant response can significantly reduce the time it takes for medical assistance to arrive, potentially saving lives.

User Interface: An intuitive and user-friendly interface allows riders to access real-time Location coordinates and their personal information such as name, contact number, blood group, etc. This interface is designed for ease of use, ensuring accessibility for all riders,

regardless of their technical expertise. The helmet also enables parents to monitor their children's riding behaviour remotely. This innovative feature promotes responsible riding and enhances safety, particularly for younger or less experienced riders.

Environmental Impact: The Smart Helmet contributes to environmental sustainability by encouraging safer and more responsible riding practices, which can reduce fuel consumption and emissions. By alerting riders when they exceed speed limits and promoting smoother riding behaviours, the helmet helps in minimizing unnecessary acceleration and braking, leading to reduced carbon emissions and a smaller environmental footprint.

Scalability and Adaptability: The Smart Helmet's design is scalable and adaptable to various types of motorcycles and riding environments. Whether used in urban settings, rural areas, or challenging terrains, the helmet's technology can be tailored to meet the specific needs of different riders. This adaptability ensures that the helmet remains effective and relevant in diverse riding conditions, offering a long-term solution for enhancing motorcycle safety.

REQUIREMENT ANALYSIS

2.1 Literature Survey

2.1.1 Related Work

The IoT-based Smart Helmet project leverages multiple theoretical frameworks to enhance the safety of two-wheeler riders. Key among these is Sensor Integration Theory, which involves the use of various sensors, such as accelerometers and Gyroscope, to detect and respond to safety-related events like crashes or speeding. This is supported by Real-Time Data Processing Theory, ensuring that the data collected is immediately analysed, enabling prompt emergency alerts to medical facilities, and thereby reducing response times in critical situations.

The project also incorporates Safety and Risk Management Theory to provide preventive alerts and reduce accident risks. Parental Control Theory further extends the system's functionality by allowing guardians to monitor trip data, promoting safe riding practices among younger or less experienced riders. These combined theories form the foundation for creating a smart helmet that significantly improves rider safety and emergency response.

2.1.2 Research Gaps of Existing Literature

The project addresses several limitations identified in previous research. It enhances alert reliability by incorporating multiple communication channels, such as SMS, app notifications, and audible alarms, to overcome issues like unreachable contacts or silent mode disruptions. By integrating multiple sensors, including accelerometers and Gyroscope, it improves crash detection accuracy, reducing false positives from road irregularities. Real-time location tracking is integrated through user's smartphone's GPS or GPRS, ensuring immediate and precise location data for emergency response teams. Additionally, the system automates emergency alerts to nearby hospitals or medical services, ensuring swift aid without relying solely on personal contacts. This comprehensive approach

addresses key challenges from earlier studies, providing a more reliable and effective safety solution.

2.1.3 Research Findings for Existing Literature

Table 2: Research Findings

S. No	Roll Number	Name	Paper Title	Tools / Technology	Findings	Citation
1.	10211748	Lovish Bansal	IoT based Smart Helmet for Bike Rider Safety	Arduino, GPS module, GSM module, Alcohol sensor, Vibration sensor.	Detects accidents and sends location to emergency contacts. Prevents drunk driving. Ensures helmet wearing before starting the bike.	(V. R. Ch , 2023) [2]
			An IoT Based Intelligent and Smart Helmet for Bike Riders Using Arduino	Arduino, Bluetooth module, Alcohol sensor, Accelerometer, GPS module.	Detects accidents and sends alerts, Monitors rider's alcohol consumption, Provides real-time location tracking.	(Rahul Kamdi , Dec 24, 2024) [3]
			Design & Implementation of IoT Based Smart Helmet for Road Accident Detection	Raspberry Pi, Accelerometer, GPS module, GSM module, Pressure sensor	Detects accidents through sudden changes in acceleration, Sends accident location to emergency services,	(M. E. Alim , 2020) [4]

					Ensures proper helmet wearing	
2.	102117157	Soham Kumar	5G & IoT Based Reporting & Accident Detection System to Deliver First Aid Box Using Unmanned Aerial Vehicle	5G network, Drones (UAV), GPS module, Accelerometer, Cloud platform	Rapid accident detection and reporting, Delivers first aid supplies via drone, Utilizes 5G for faster data transmission	(Monagi H. Alkinani , 2021) [5]
			IoT Based Smart Helmet and Accident Identification System	Arduino, GPS module, GSM module, Accelerometer, Alcohol sensor	Detects accidents and sends alerts, Prevents drunk driving, Provides real-time location information	(Md. Atiqur Rahman , June7 ,2020) [7]
3.	102117184	Aryan Tyagi	SMART HELMET SAFETY SYSTEM USING ATMEGA 32[8]	ATmega32 microcontroller, Alcohol sensor, Vibration sensor, GSM module	Ensures helmet wearing, Detects alcohol consumption, Sends accident alerts to predefined numbers	(Mangesh Jadhawar , May 5 ,2016) [8]
			A prototype of IoT-based smart system to support motorcyclists safety[9]	Wearable sensors, Bluetooth Low Energy (BLE), Smartphone application, Cloud server, Machine	Real-time monitoring of rider's physiological state, Detection of dangerous riding behaviors, Alert system	(P Prasetyawan , 2021) [9]

				learning algorithms	for potential risks, Data analysis for long-term safety improvements	
4.	102117149	Garv Bundel	IoT based Smart Helmet with Motorbike Unit for Enhanced Safety [10]	Arduino microcontroller, GPS module, GSM module, Alcohol sensor, Accelerometer, RF transmitter and receiver	Ensures helmet wearing before bike ignition, Detects alcohol consumption and prevents drunk driving, Accident detection and automatic emergency notification, Real-time location tracking, Bike anti-theft system	(P. Pathak , 2020)[10]

2.1.4 Problems Identified

The absence of real-time alerts to nearby hospitals and the lack of multiple emergency contact options pose a significant risk in critical situations. Also, the current IoT-based smart helmet systems face several challenges impacting their effectiveness. Issues include the risk of alert SMS being missed if the pre-defined emergency contact is busy, unreachable, or if the alert device is on silent mode or out of battery. Furthermore, reliance on single sensors for crash detection and alcohol detection reduces system reliability, as these sensors can be easily tampered with or blocked. Inaccuracies in detecting sudden jerks caused by speed breakers or potholes are also common, leading to false alarms or undetected crashes. Limited smartphone functionality during or after a crash, due to damage or

weak signal coverage, hinders data collection and response time. Other limitations include the absence of a robust database system with web support, dependency on specific sensors that may fail under varying conditions, and the lack of GPS and connectivity capabilities, which are essential for precise location tracking and enhanced connectivity.

2.1.5 Survey of Tools and Technology Used

The smart helmet relies on microcontrollers like the Arduino Nano and ATMEGA 32, which serve as the core units for data processing. They control sensors, manage data communication between helmet and bike modules, and monitor critical safety conditions. RF Transmitter and Receiver (e.g., nRF24L01) facilitate wireless communication between helmet and bike units, while GSM and GPS modules play a vital role in emergency notifications. The GSM module sends accident alerts and updates to pre-programmed contacts, while the GPS data tracks the vehicle's location, crucial for accident detection and real-time updates. IoT modules connect helmet data to cloud-based platforms, enabling edge servers to relay accident information to nearby hospitals and update the vehicle and hospital databases. Web server integration ensures secure data logging and management. The system is programmed using Arduino IDE and Blynk App for user interface development.

2.1.6 Summary

The IoT Smart Helmet is a groundbreaking innovation designed to redefine motorcycle safety by integrating IoT technologies, real-time data analytics, and seamless connectivity. Unlike traditional helmets, which offer only passive protection, this device actively monitors the rider's environment and reacts to critical situations, ensuring enhanced safety and connectivity. It bridges the gap between conventional safety gear and modern technological capabilities, positioning itself as an indispensable tool for motorcyclists.

The IoT Smart Helmet works seamlessly with existing systems to create a comprehensive safety network. It complements traditional emergency response systems by automatically sending alerts to nearby medical services during a crash, including precise GPS coordinates. The helmet also provides a sophisticated family

notification system, immediately informing pre-defined contacts about emergencies. Moreover, it integrates with riders' smartphones or web applications, allowing users to access real-time data, monitor trips, and customize safety features. This seamless integration not only enhances the rider's safety but also promotes transparency and accountability, particularly for younger or inexperienced riders.

The helmet offers a range of advanced features that make it a unique and comprehensive safety solution. The crash detection system uses accelerometers and gyroscopes to detect significant impacts and trigger an emergency response. The crash detect alert system issues audible warnings when a rider exceeds predefined thresholds, encouraging safer riding habits. Furthermore, the helmet supports real-time connectivity, enabling riders to synchronize data with their smartphones or access it directly from the helmet's network.

2.2 Software Requirement Specification

2.2.1 Introduction

2.2.1.1 Purpose

The IoT Smart Helmet project has been designed to address the critical need for enhanced safety and real-time monitoring for motorcyclists in an increasingly urbanized and high-traffic world. By leveraging advanced technologies such as IoT connectivity, GPS tracking (from smartphone), and sensor integration, this project aims to significantly improve rider safety by providing immediate alerts and automated emergency response in the event of a crash or accident.

Some of the key objectives include reducing the time it takes for emergency services to respond to accidents by automatically notifying the nearest medical facility with accurate location data. This can potentially save lives by ensuring timely medical intervention.

Further supporting the overall objective of road safety, the IoT Smart Helmet helps to minimize the risks associated with motorcycling by providing continuous data on rider conditions and environmental factors. The system's ability to transmit real-

time data ensures that safety measures are always in place, reducing the likelihood of accidents and enhancing the overall riding experience.

In short, the IoT Smart Helmet is a system designed to revolutionize rider safety and accident response through the use of innovative technology. This aims to save lives, improve emergency response times, and contribute to safer roads and a more secure riding experience for motorcyclists.

2.2.1.2 Intended Audience and Reading Suggestions

The IoT-based Smart Helmet project report is designed to cater to a diverse audience. Students in fields such as computer science, electrical engineering, and mechanical engineering who are interested in IoT applications, embedded systems, and safety technology will find this report particularly beneficial. It provides insights into the integration of hardware and software components to create an innovative safety device. Researchers focusing on IoT, smart devices, safety systems, and human-computer interaction will find valuable information in this report. The detailed documentation of the development process, including the crash detection algorithm and sensor integration, offers a foundation for further research and development. Professionals working in the fields of automotive safety, wearable technology, and IoT product development will benefit from understanding the practical implementation of a smart safety device. The report highlights challenges, solutions, and potential improvements in the design and deployment of IoT-based systems. Entrepreneurs and startups interested in developing smart safety products for the automotive industry can gain insights into market gaps, user needs, and the technical aspects of creating a smart helmet. The report outlines potential future enhancements and scalability, offering a roadmap for product development. Organizations and authorities responsible for setting safety standards and regulations in the automotive industry may find the report useful in understanding the capabilities of IoT technology in enhancing rider safety. It may also inform discussions on new regulations or standards for smart safety devices. Parents and guardians concerned about the safety of young or inexperienced riders can use the report to understand how the Smart Helmet can help monitor and promote safe riding practices.

2.2.1.3 Project Scope

The IoT-based Smart Helmet project aims to bridge this gap by developing an intelligent, connected helmet that enhances rider safety through a combination of real-time monitoring, automated emergency response, and behaviour analysis.

This project focuses on creating an embedded system capable of detecting crashes in real time using accelerometers and gyroscopes, with GPS (from smartphone) integration to accurately determine the location of accidents. Upon detecting a crash, the system will automatically send emergency alerts, including the rider's coordinates, to nearby medical services. In addition to crash detection, the helmet will continuously monitor the rider's behaviour and condition, with a built-in crash alert system to warn the rider when they experience a jerk that exceeds the threshold. A web-based interface will be developed to allow parents or guardians to monitor riding behaviour and location promoting responsible riding practices. The project will also ensure that the user interface is intuitive and easily accessible, providing a seamless experience across mobile and web platforms. Furthermore, the scope includes laying the groundwork for future scalability, with potential enhancements such as integration with additional IoT devices, expanded sensor capabilities, and broader data analytics, making the solution adaptable for various transportation modes and user needs. The project will offer an effective solution now and suitable adaptation of technology with a strong focus on user experience and scalability.

2.2.2 Overall Description

2.2.2.1 Product Perspective

The IoT Smart Helmet is a ground-breaking innovation designed to redefine motorcycle safety by integrating IoT technologies, real-time data analytics, and seamless connectivity. Unlike traditional helmets, which offer only passive protection, this device actively monitors the rider's environment and reacts to critical situations, ensuring enhanced safety and connectivity. It bridges the gap between conventional safety gear and modern technological capabilities, positioning itself as an indispensable tool for motorcyclists.

The IoT Smart Helmet works seamlessly with existing systems to create a comprehensive safety network. It complements traditional emergency response systems by automatically sending alerts to nearby medical services during a crash, including precise GPS coordinates. Moreover, it integrates with riders' smartphones or web applications, allowing users to access real-time data, monitor trips, and customize safety features. This seamless integration not only enhances the rider's safety but also promotes transparency and accountability, particularly for younger or inexperienced riders.

The helmet offers a range of advanced features that make it a unique and comprehensive safety solution. The crash detection system uses accelerometers and gyroscopes to detect significant impacts and trigger an emergency response. Real-time location monitoring captures the user's precise coordinates. The crash alert system issues audible warnings when a rider experiences a jerk/shock that exceeds predefined thresholds. Furthermore, the helmet supports real-time connectivity, enabling riders to synchronize data with their smartphones or access it directly from the helmet's network.

2.2.2.2 Product Features

1. Automatic Transmission of an Emergency Signal:

Upon detecting a crash through its integrated sensors (accelerometer, gyroscope, etc.), the IoT Smart Helmet will automatically trigger an emergency signal. This signal is sent to the nearest medical facilities, prioritizing proximity over other factors to ensure the quickest possible response time.

Transmission of Coordinates and Essential Rider Information:

Along with the emergency signal, the helmet transmits precise GPS coordinates of the crash location, along with essential rider information such as blood type, known medical conditions, and emergency contact details. This data is crucial for medical teams to prepare in advance and provide appropriate care, potentially saving lives through rapid response and treatment.

2. Medical Notification System

Instant Dispatch of Emergency services:

When a crash is detected, the system instantly sends an emergency alert to emergency services. This message includes the rider's current location and a brief information about the user's condition situation.

3. Trip Monitoring and Report

GPS-based Trip Data Recording:

The helmet utilizes GPS technology to monitor and record rider's location data. This data is then stored on a secure web platform that is easily accessible to the rider or authorized users.

4. Crash Alert System

Distinctive Beeping Sound for Crash Alerts:

The helmet is equipped with a Crash alert system that generates a distinctive beeping sound whenever the rider experiences a jerk. This auditory alert is designed to be noticeable without being overly disruptive.

2.2.3 External Interface Requirements

2.2.3.1 User Interfaces

The User Interface (UI) of the IoT Smart Helmet is designed with a focus on simplicity, clarity, and ease of use to ensure that riders, hospitals, and guardians can effectively interact with the system. The UI is primarily accessed through a dedicated website that serves as the central hub for monitoring and managing the helmet's features.

The rider's data is presented in a clean, intuitive dashboard that ensures information is easy to comprehend, even at a moment's notice. The UI includes large, clearly labelled buttons and visual cues to assist riders in accessing important functions.

For hospitals, the UI offers functionality to monitor incoming emergency alerts from riders in need of assistance. The interface provides real-time location data of the rider, along with critical information such as the rider's emergency contact

details, blood group, and crash severity, enabling hospitals to prepare for immediate response. Hospitals can log in securely to view and manage alerts, ensuring prompt and effective action during emergencies.

The UI is responsive and accessible on various devices, including desktops, tablets, and smartphones. This flexibility ensures that users can interact with the system regardless of their device, offering a seamless experience across platforms. Additionally, the UI supports real-time notifications and alerts, ensuring that critical information is conveyed promptly to both riders and hospitals, enhancing overall safety and connectivity.

2.2.3.2 Hardware Interfaces

The Hardware Interfaces of the IoT Smart Helmet are critical to its functionality, ensuring seamless communication between the helmet's embedded sensors, GPS module, and external devices. These interfaces facilitate the integration of various components that work together to enhance rider safety and provide real-time data to users.

At the core of the helmet's hardware system is the sensor suite, which includes accelerometers, gyroscopes, and other motion-detecting devices. These sensors are directly interfaced with a microcontroller, which processes the data to detect events such as crashes or sudden impacts. The microcontroller is designed to communicate efficiently with these sensors, ensuring low latency and accurate detection of critical incidents.

The helmet also incorporates a GPS module, which is interfaced with the microcontroller to provide real-time location tracking. This module continuously relays geographical coordinates, which are essential for features such as trip monitoring and emergency assistance. The GPS data is processed and sent via a wireless communication interface to a cloud server, where it can be accessed by the user through the associated web interface.

For connectivity, the helmet is equipped with communication modules for Wi-Fi capabilities. These modules are interfaced with the microcontroller to transmit data to external devices, such as smartphones or emergency services, in the event of an incident. Wi-Fi module provides the ability to send emergency notifications and trip data to remote servers.

Power management is another key aspect of the hardware interface. The helmet is powered by a rechargeable battery, which is integrated with a power management unit (PMU) to ensure consistent power supply to all components. The PMU interfaces with the microcontroller to monitor battery levels and optimize power usage, extending the operational life of the helmet between charges.

Overall, the hardware interfaces of the IoT Smart Helmet are meticulously designed to ensure reliable operation, seamless communication between components, and efficient power management, all of which contribute to the device's effectiveness in enhancing rider safety and connectivity.

2.2.3.3 Software Interfaces

The Software Interfaces of the IoT Smart Helmet is crucial for managing the interaction between the helmet's hardware components and external applications. These interfaces are designed to ensure that data collected by the helmet's sensors is processed, transmitted, and displayed accurately and efficiently, providing users with a seamless experience.

The core of the software interface is the firmware running on the helmet's microcontroller. This firmware is responsible for collecting data from the various sensors, processing it in real time, and triggering appropriate actions based on the conditions detected. For instance, in the event of a crash, the firmware instantly processes the sensor data, activates the communication module, and sends out emergency alerts with the rider's GPS location.

The helmet communicates with external devices and cloud services through Application Programming Interfaces (APIs). These APIs are designed to facilitate

the secure transmission of data from the helmet to the user's profile on the website. The APIs enable features like real-time trip tracking, which allows the user to monitor their journey through a web interface. The API architecture ensures that data is transmitted efficiently, with minimal latency, and can handle multiple types of requests, such as fetching historical trip data or configuring alert settings.

The IoT Smart Helmet also integrates with a cloud-based server through its software interface. The server acts as a central repository where all trip data, sensor logs, and user settings are stored. The cloud interface is built to handle large volumes of data, offering scalability as more helmets are deployed and more data is generated. This server interface also supports data analytics, allowing users to generate reports and insights about their riding habits. The server communicates back to the helmet or the website, ensuring that the information is always synchronized across devices.

Finally, the software interface includes a web application interface that allows users to interact with the helmet's features. This interface is designed to be user-friendly, providing access to real-time data, precise location coordinates, alerts in case of a crash and settings through a graphical user interface. The website interfaces with the helmet's firmware and cloud server via the aforementioned APIs, ensuring that users can monitor and control their helmet's functionality from their smartphone or computer with ease.

In summary, the software interfaces of the IoT Smart Helmet are designed to ensure smooth communication between the helmet's hardware, cloud services, and web applications. These interfaces are vital for delivering the helmet's safety features, providing real-time data, and enabling users to interact with their devices securely and efficiently.

2.2.4 Other Non-functional Requirements

2.2.4.1 Performance Requirements

The performance requirements for the IoT Smart Helmet focus on ensuring quick and efficient system operations, crucial for real-time safety applications. The helmet's software must provide low latency in crash detection and emergency signal transmission to guarantee prompt responses in critical situations. Additionally, the system should be optimized to deliver rapid feedback, especially in crash alerts and trip data retrieval, ensuring that the rider is constantly informed and protected without delays.

In addition to the helmet's performance requirements, the web application's performance requirements are equally critical. The web application, which displays data received from the helmet module, must be optimized for fast loading times. It should efficiently handle and display real-time data quickly and without lag. Furthermore, the application should be scalable to manage increasing amounts of data as more users utilize the helmet system, ensuring consistent performance even under heavy loads. This will enable users to monitor and analyse their riding activities seamlessly, contributing to both safety and user satisfaction.

2.2.4.2 Safety Requirements

The safety requirements are centred on the helmet's ability to enhance rider protection through real-time monitoring and emergency response capabilities. The system should accurately detect crashes using data from accelerometer and gyroscope sensors and immediately transmit to the emergency services along with the rider's GPS coordinates.

2.2.4.3 Security Requirements

For security requirements, it is essential to implement robust measures to protect user data and maintain privacy. Moreover, user authentication mechanisms must be in place to ensure that only authorized individuals can access sensitive information, such as trip data and emergency contact details. This focus on security helps in maintaining user trust and the overall integrity of the system.

2.3 Cost Analysis

Table 3: Cost Analysis

Product	Quantity	Cost (Rs.)
Uno SMD (arduino uno, jumper wires)	1	500rs
Node MCU ESP8266	1	345rs
Node MCU ESP32	1	320rs
MPU-6050 module (3-axis gyroscope + 3-axis accelerometer)	2	374rs
GPS Module Neo-C6M	2	790rs
LCD Display	1	127rs
Buzzer, Switches	5, 2	63rs
Gyroscope	2	129rs
Buzzers	5	104rs
Helmet	1	400rs
Breadboard, Jumper wires, LED	1, 60, 2	228rs
GSM module	1	300rs
Total	Rs. 3700	

2.4 Risk Analysis

The risk analysis for the IoT-based Smart Helmet project aims to identify and evaluate potential risks that could impact the success of the project. These risks are categorized into three main areas: hardware integration, software performance, and deployment timelines. Below is an analysis of these risks, considering their probability and impact, along with recommended strategies for mitigating their effects.

Technical Risks:

1. Hardware Integration:

- **Risk:** There is a possibility of incompatibility or functionality issues arising from the integration of various hardware components, such as sensors (gyroscopes, accelerometers), communication modules (Node MCU), and the GPS data through user's smartphone. These issues could affect the helmet's overall performance and reliability.
- **Mitigation:** To mitigate this risk, thorough testing and validation of hardware components should be conducted. Compatibility checks between different modules are essential to ensure seamless integration. Additionally, prototype iterations should be tested under various conditions to identify and address any potential hardware failures early in the development process.

2. Software Performance:

- **Risk:** The software responsible for processing data from sensors and managing real-time communication may encounter performance issues, such as delays in data processing, inaccuracies in sensor readings, or failures in triggering emergency responses. These issues could undermine the helmet's effectiveness and user trust.
- **Mitigation:** Rigorous performance testing and optimization of the software are critical. This includes stress testing the software under different scenarios to identify potential bottlenecks or vulnerabilities. Implementing robust error-handling mechanisms and continuous monitoring during the development phase will help ensure that the software performs reliably under various conditions.

3. Deployment Delays:

- **Risk:** The project may face delays in deployment due to unforeseen technical problems, logistic difficulties, or supply chain disruptions. These delays could extend the project timeline and increase costs, potentially jeopardizing the project's success.
- **Mitigation:** To mitigate deployment risks, a detailed project timeline with built-in buffer periods should be established. Contingency plans should be developed to address potential setbacks, such as alternative suppliers for components or backup plans for key project milestones. Regular progress reviews and risk assessments will help identify and address potential delays before they become critical.

Risk Assessment: The likelihood of each risk occurring is weighed against its potential impact on the project. This risk assessment allows the project team to prioritize serious risks and allocate resources accordingly to minimize their impact.

- **High-Priority Risks:** Hardware integration issues and software performance failures are considered high-priority risks due to their direct impact on the helmet's functionality and safety. These risks require immediate attention and proactive mitigation strategies.
- **Moderate-Priority Risks:** Deployment delays are classified as moderate-priority risks, with their impact primarily affecting the project timeline and budget. While these risks are significant, they can be managed with proper planning and contingency measures.

The risk analysis serves as a proactive measure to identify and address potential challenges that could arise during the development and deployment of the IoT-based Smart Helmet. By forewarning the project team of possible problems and outlining risk mitigation strategies, the analysis aims to ensure that the Smart Helmet is reliable, secure, and efficient in meeting user needs with minimal disruptions to the project's progress.

METHODOLOGY ADOPTED

3.1 Investigation Techniques

Table 4: Investigation Techniques

S.No.	Investigate Project Techniques	Investigative Description Techniques	Investigative Examples	Project
1	Literature Review	At the initial stage of this project, an extensive literature survey was conducted to review the existing research and advancements in the area of smart helmet systems, with a special focus on IoT-based safety solutions. The review covered studies related to crash detection, alcohol detection, and emergency response systems integrated within helmets. The goal was to identify effective practices, challenges, and innovative approaches relevant to designing a Smart Helmet IoT Device. By analyzing prior work, we aimed to gain insights that could guide our project's design, enhance safety features, and inform the development of improved crash detection, communication, and data processing mechanisms.	Reviewing the available research papers, articles, and case studies to identify algorithms, already existing frameworks, and systems that are relevant to other available smart helmets.	
2	Case Study Analysis	This case study analysis technique involved examining the already existing smart helmet that are available	Analysing case studies of smart helmet systems that utilize IoT and sensor	

		in the market. The analysis is focused on understanding challenges, learning from the success stories of the existing systems and studying how different technologies were integrated together.	technology for alcohol detection and automated emergency signal.
3	Technical Feasibility Study	The smart helmet project involves integrating IoT devices and communication modules to enable real-time monitoring and emergency response. The software must handle sensor input, GPS data (from user smartphone) , and emergency alerts efficiently. The smart helmet will need to function across various real-world conditions, such as different weather environments, high vibration, and temperature changes. Testing under various scenarios, including speed variations and collision impact simulations, will help detect potential issues and improve robustness. Regulatory compliance with safety standards and telecommunication laws is critical, especially for wireless transmission (GSM or GPRS). The project must also ensure data privacy and security, given that personal information is involved.	Testing libraries like the MPU6050 and WiFiManager, evaluating different sensors and microcontrollers, and simulating various real-world scenarios to ensure system reliability and scalability.
4	Prototyping and Simulation	Prototyping and simulation were used to test and refine the system in a controlled environment before full-	Developing prototypes of the smart helmet module simulating different safety

		scale deployment. These techniques helped in early detection of issues, iterative improvement, performance benchmarking, and user interface testing.	scenarios, and testing user interface designs to ensure they are intuitive and user-friendly.
--	--	--	---

3.2 Proposed Solution

The IoT Smart Helmet represents a ground breaking innovation in rider safety, integrating advanced IoT technologies with traditional safety gear to provide an intelligent, responsive, and data-driven approach to motorcycle riding. This proposed solution is not just a technological enhancement but a comprehensive rethinking of how safety can be achieved through real-time data monitoring, analysis, and communication. The core of the IoT Smart Helmet lies in its ability to monitor various aspects of the rider's environment and condition in real-time. By embedding a suite of advanced sensors—such as accelerometers and gyroscopes—the helmet is capable of tracking critical metrics like speed, orientation, and impact forces.

In the event of a crash, the helmet's integrated sensors can quickly detect the impact, triggering an automated emergency response sequence. This immediate recognition of an accident is critical, as the time between an incident and the arrival of emergency services can be the difference between life and death. The helmet will instantly transmit the rider's GPS coordinates through the rider's smartphone and a distress signal to nearby medical facilities and pre-defined emergency contacts, significantly reducing response times.

The IoT Smart Helmet goes beyond the traditional scope of personal safety devices by incorporating an automated emergency assistance system. Upon detecting a significant impact or other dangerous conditions, the helmet's on-board microcontroller communicates with an external server via a Wi-Fi or cellular connection. This connection can be established either through a smartphone paired with the helmet or via a built-in network module. Once an emergency is detected, the server relays the distress signal and

GPS location to the nearest emergency services and sends an alert to the rider's family or emergency contacts.

To enhance rider safety, the IoT Smart Helmet includes a crash alert system. This feature leverages the helmet's hardware modules to compare the jerk experienced by the rider against a pre-defined threshold. If the rider exceeds these limits, the helmet emits an audible alert.

The crash alert system is particularly valuable in areas with frequently changing speed limits, such as urban environments or school zones. By providing real-time feedback, the helmet assists riders in adjusting their speed appropriately, reducing the risk of collisions.

In today's connected world, the ability to access and share information quickly is essential. The IoT Smart Helmet addresses this need by integrating seamless connectivity options that allow data to be transmitted, stored, and accessed effortlessly. The helmet can connect to a smartphone through the internet via Wi-Fi or a built-in cellular module. This connectivity enables the helmet to sync with cloud-based services, where trip data, emergency alerts, and other information are securely stored.

Users can access this data through a dedicated web portal, providing a centralized platform for monitoring helmet activity, reviewing trip logs, and configuring system settings. The platform is designed to be intuitive and user-friendly, with clear visualizations and easy-to-navigate menus.

The IoT Smart Helmet is designed with scalability in mind, allowing for future expansions and enhancements. As new technologies emerge, the helmet can be upgraded with additional sensors, improved connectivity options, and more advanced data analysis algorithms.

The IoT Smart Helmet offers a comprehensive solution to the challenges of motorcycle safety in the modern age. By combining real-time monitoring, emergency assistance, and advanced data analytics, the helmet not only protects riders but also empowers them with valuable insights into their riding habits. The seamless integration of connectivity and data

access ensures that users can stay informed and make data-driven decisions to enhance their safety on the road.

This proposed solution not only addresses current safety concerns but also lays the groundwork for future innovations in rider protection. As the IoT Smart Helmet evolves, it will continue to set new standards for motorcycle safety, providing riders with the tools they need to navigate the roads with confidence and peace of mind.

3.3 Work Breakdown Structure (WBS)

During the development of the Smart Helmet, a number of key modules or products can be identified that present a critical component of the solution in its entirety. The WBS produces a clear roadmap for the project and explains all tasks with deliverables associated to ensure that the objectives of the project are realized.

1. Project Management

1.1 Project Planning

- 1.1.1 Define project scope
- 1.1.2 Allocate resources and assign tasks

1.2 Documentation

- 1.2.1 Create a project charter
- 1.2.2 Prepare requirement specifications
- 1.2.3 Develop user manual and technical documentation

1.3 Monitoring & Control

- 1.3.1 Track progress and adjust schedules
- 1.3.2 Conduct regular team meetings

2. Hardware Development

2.1 Sensor Integration

- 2.1.1 Research and select appropriate sensors (accelerometer, gyroscope, etc)
- 2.1.2 Procure sensors and other hardware components

- 2.1.3 Integrate sensors with the NodeMCU microcontroller
- 2.1.4 Test sensor data collection and accuracy

2.2 Design circuit schematic for helmet integration

2.3 Determine power requirements for all components

3. Firmware Development

3.1 NodeMCU Programming

- 3.1.1 Set up a development environment (VS Code with PlatformIO/Arduino IDE)
- 3.1.2 Write code for sensor data acquisition
- 3.1.3 Implement Access Point (AP) functionality
- 3.1.4 Develop code for emergency detection and alert system

3.2 Communication Protocols

- 3.2.1 Implement Wi-Fi communication for data transmission
- 3.2.2 Develop API endpoints for sending data to the server
- 3.2.3 Ensure secure data transmission (e.g., HTTPS)

3.3 Testing & Debugging

- 3.3.1 Conduct unit testing of firmware modules
- 3.3.2 Debug and resolve firmware issues
- 3.3.3 Perform integration testing with the hardware

4. Backend Development

4.1 Database Setup

- 4.1.1 Choose a database
- 4.1.2 Design database schema for storing helmet data
- 4.1.3 Set up and configure the MongoDB instance

4.2 Server-Side Development

- 4.2.1 Choose a backend framework (e.g., Node.js with Express, Python with Flask)
- 4.2.2 Develop API for receiving and storing data from the helmet
- 4.2.3 Implement logic for emergency alerts and family notifications

4.3 Data Processing & Analytics

- 4.3.1 Develop algorithms for data analysis and reporting
- 4.3.2 Integrate data analytics with the frontend interface

5. Frontend Development

5.1 UI/UX Design

- 5.1.1 Create wireframes and prototypes
- 5.1.2 Design a user-friendly interface for data visualization

5.2 Web Development

- 5.2.1 Set up web framework (e.g., React, Angular)
- 5.2.2 Implement frontend components for data display

5.3 Integration with Backend

- 5.3.1 Develop API integration for real-time data fetching
- 5.3.2 Implement user authentication and data privacy features
- 5.3.3 Perform end-to-end testing of the web application

6. Testing and Quality Assurance

6.1 Hardware Testing

- 6.1.1 Perform field tests for sensor accuracy and reliability
- 6.1.2 Validate emergency response triggers and notification systems

6.2 Software Testing

- 6.2.1 Conduct unit tests on individual software modules
- 6.2.2 Perform integration tests between hardware, firmware, and backend
- 6.2.3 Carry out system testing and user acceptance testing

6.3 Debugging & Optimization

- 6.3.1 Identify and resolve bugs and performance issues
- 6.3.2 Optimize code for efficiency and scalability
- 6.3.3 Ensure compliance with safety and security standards

7. Project Closure

7.1 Final Documentation

- 7.1.1 Compile all project documentation, including design specs and test results
- 7.1.2 Document lessons learned and best practices
- 7.1.3 Archive project materials for future reference

3.4 Tools and Technology

For the smart helmet project, integrating key technologies and hardware components is essential. The central microcontroller, such as NodeMCU esp8266 or NodeMCU esp32, processes sensor data and controls other modules. Wireless communication is achieved through RF transmitters and receivers. The GPS data from the rider's smartphone tracks real-time location, stores trip data, and provides coordinates for emergency alerts. The crash alert is then displayed on the Hospital interface. Sensors like accelerometers and gyroscopes detect crashes and monitor riding patterns, with crash detection triggering alerts to hospitals and sending GPS data for a quick response. A web interface allows guardians to monitor the rider's status, while backend services process sensor data into actionable insights, enhancing rider safety with a user-friendly platform.

DESIGN SPECIFICATIONS

4.1 System Architecture

4.1.1 Block Diagram

Central hub that connects all other modules. Receives data from the rider's module, such as crash detection. Displays information to both the rider and hospital, like crash detection and crash location. Sends crash location and rider information to the hospital interface if an accident is detected. Sends crash alerts to the hospital interface. Receives display information and alerts on the hospital interface. Displays relevant information like the rider's basic details and crash location. Receives crash location and rider information when an accident is detected. Connected to the Interface for emergency alerts. Accessible by the interface's for real-time operations.

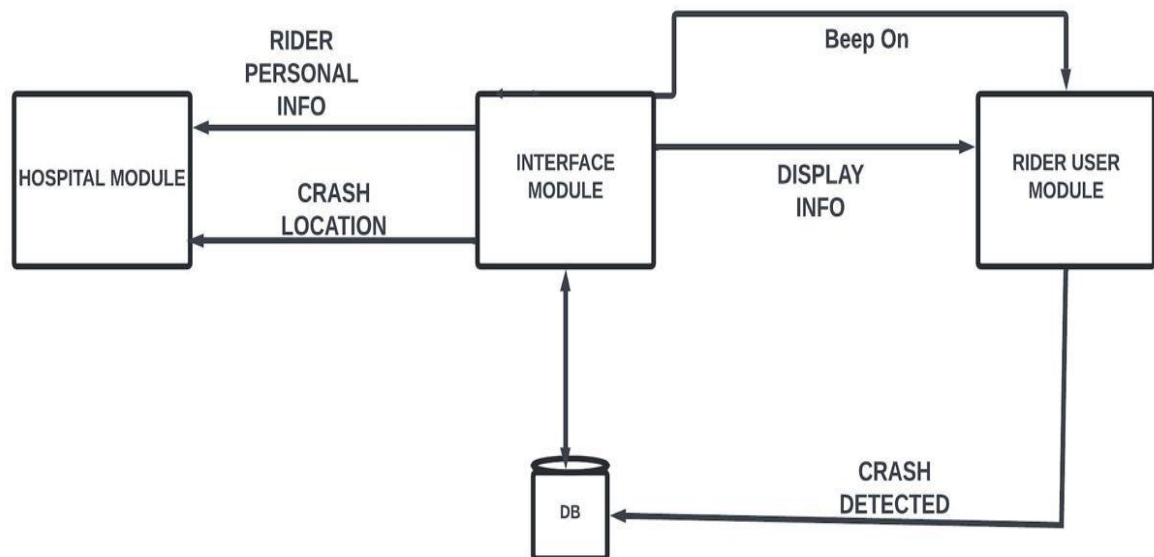


Figure 1: Block Diagram for Smart Helmet

4.1.2 MVC Architectural Tier Diagram

The proposed IoT-based smart helmet system is built on an MVC architecture that separates concerns to ensure modularity, scalability, and maintainability. The Model encompasses the Helmet IoT Device, Hospital User Interface, and Database. The Helmet Device collects rider data like tilt and location (from smartphone), while the Hospital interface shows emergency alerts and rider data. The Controller coordinates interactions between Model and View with components like AlertBeepController (managing sound alerts), and DataController (handling data flow). The **View** offers user interfaces, including a WebInterface for user management and an interface at the hospital for displaying critical information like rider details and alerts. This architecture enhances user experience with a responsive and intuitive system.

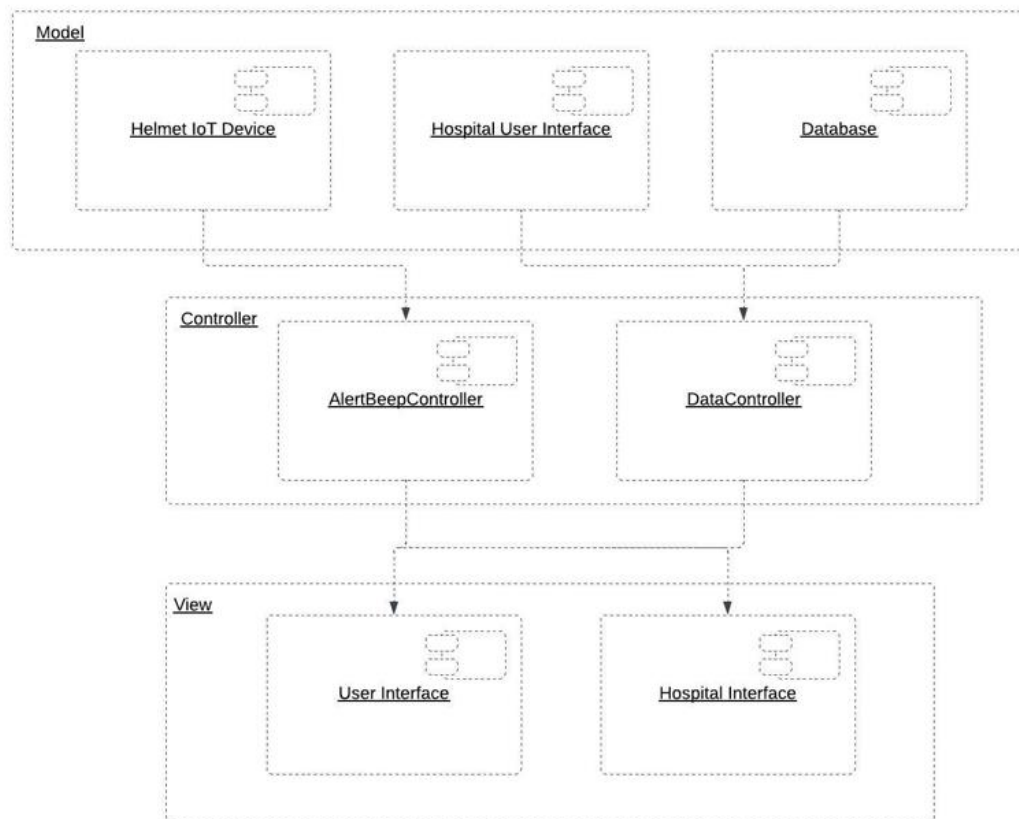


Figure 2: MVC Architectural Tier Diagram for Smart Helmet

4.2 Design Level Diagrams

4.2.1 Data Flow Diagrams (DFD)

DFD Level 0 Diagram: The IoT-based Smart Helmet system provides an overview of the system's interactions and processes. At the center is the Alert System Process, which coordinates data flow between the various components. The Helmet IoT Device sends crash information and basic rider information, while the Rider interacts by logging in and receiving alerts. In case of an emergency, the system signals the Hospital user interface. This diagram highlights the system's main processes and interactions, focusing on real-time alerts and safety features without diving into internal details.

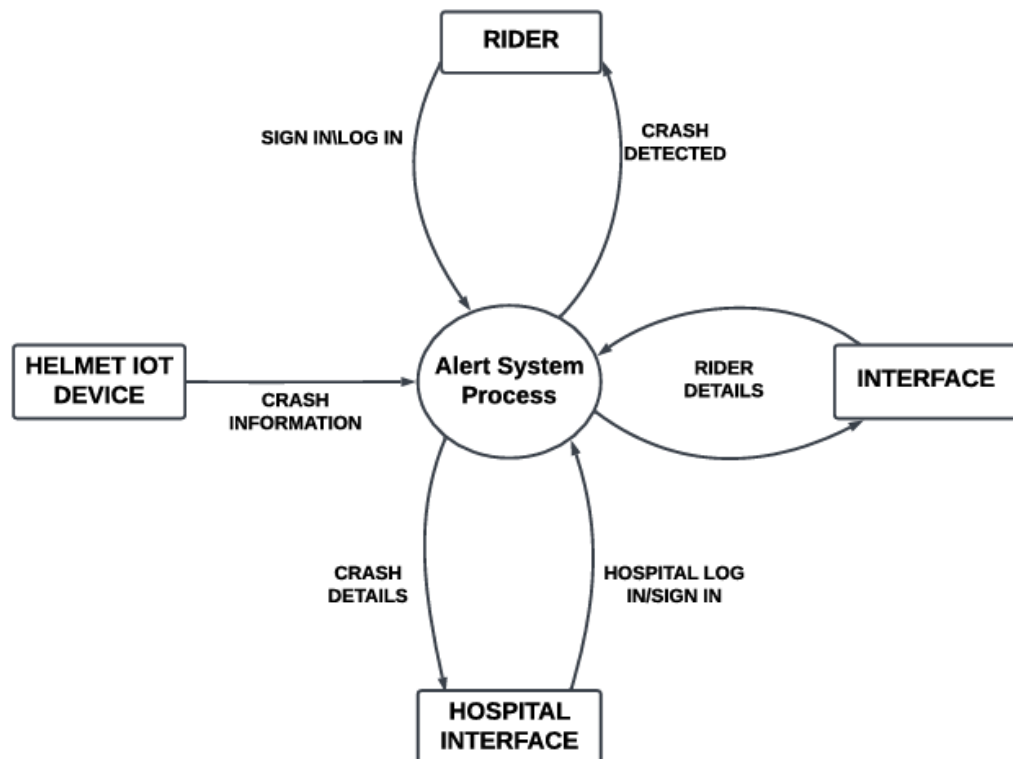


Figure 3: DFD (Level 0) for Smart Helmet

DFD Level 1 Diagram: The IoT-based Smart Helmet system details the data flow between key components. The Helmet IoT Device manages crash alerts and detects incidents, sending data to the rider and relevant entities. In case of an emergency, signals are sent to the Hospital to trigger services, while the interface provides real-time updates. This diagram breaks down interactions like crash detection, and alert generation, offering deeper insights into the system's functionality.

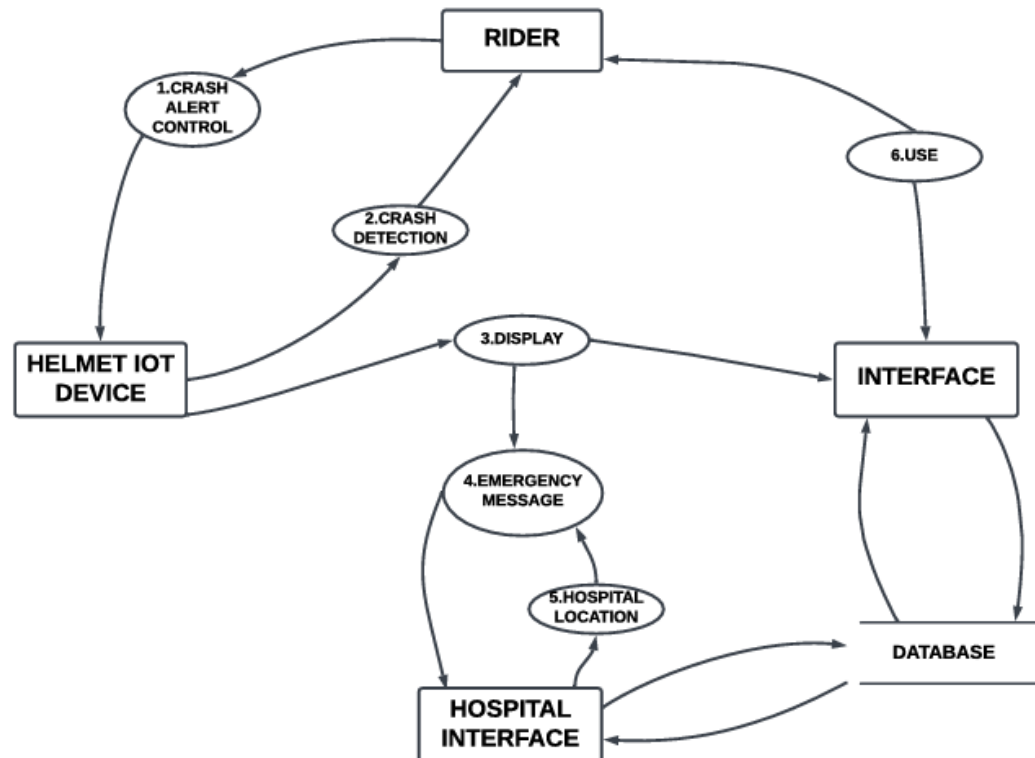


Figure 4: DFD (Level 1) for Smart Helmet

DFD Level 2 Diagram: The IoT-based smart helmet illustrates the detailed data flow within the system, focusing on interactions between the helmet IoT device, rider, hospital, and interface. It shows how the helmet detects crashes, detects false crash, and communicates with the rider and hospital through user interfaces. In the event of a crash, the system notifies the hospital, sending coordinates and the rider's user profile. This diagram provides insight into the internal data handling and response mechanisms of the smart helmet system.

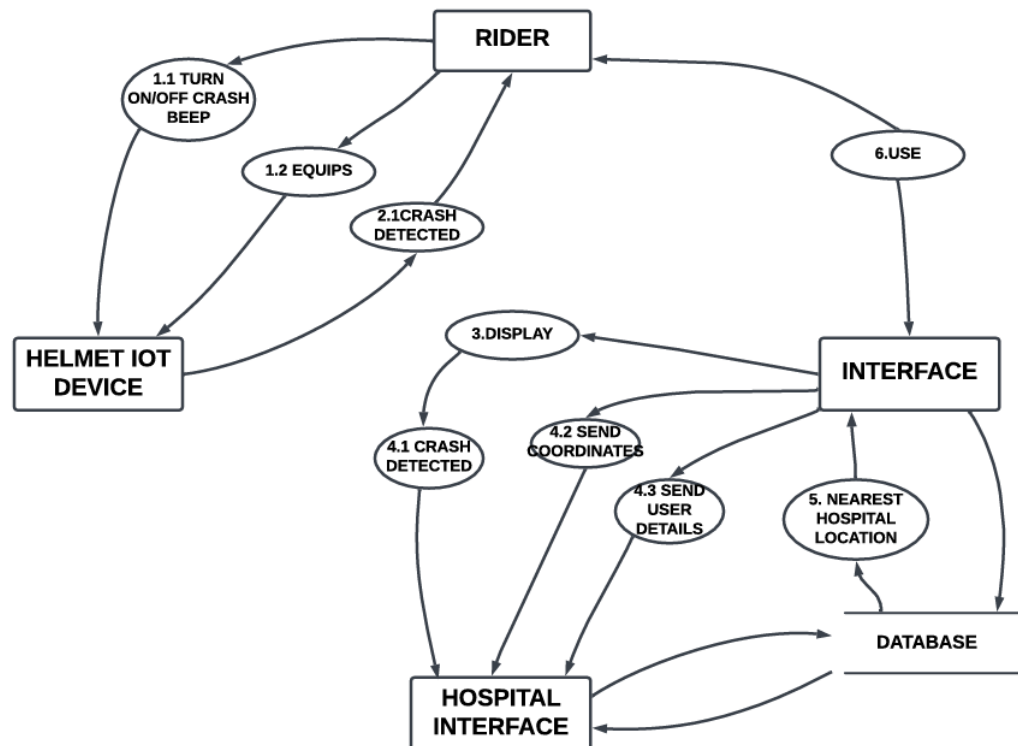


Figure 5: DFD (Level 2) for Smart Helmet

4.2.2 Entity Relationship Diagram

This ER diagram provides a visual representation of the data elements and their relationships in a smart helmet system. It helps in understanding the structure of the data, designing the database, and developing applications that interact with the system.

Relationships between entities:

- Driver has a Device.
- Driver has Parents.
- Device is at a Location.
- Device has a Battery.
- Device sends Details to a Database.
- Device is associated with a Hospital.
- Hospital has a Hospital Device.
- Hospital is at a Location.
- Hospital has Trip Info.
- Trip Info can have a Crash Location.

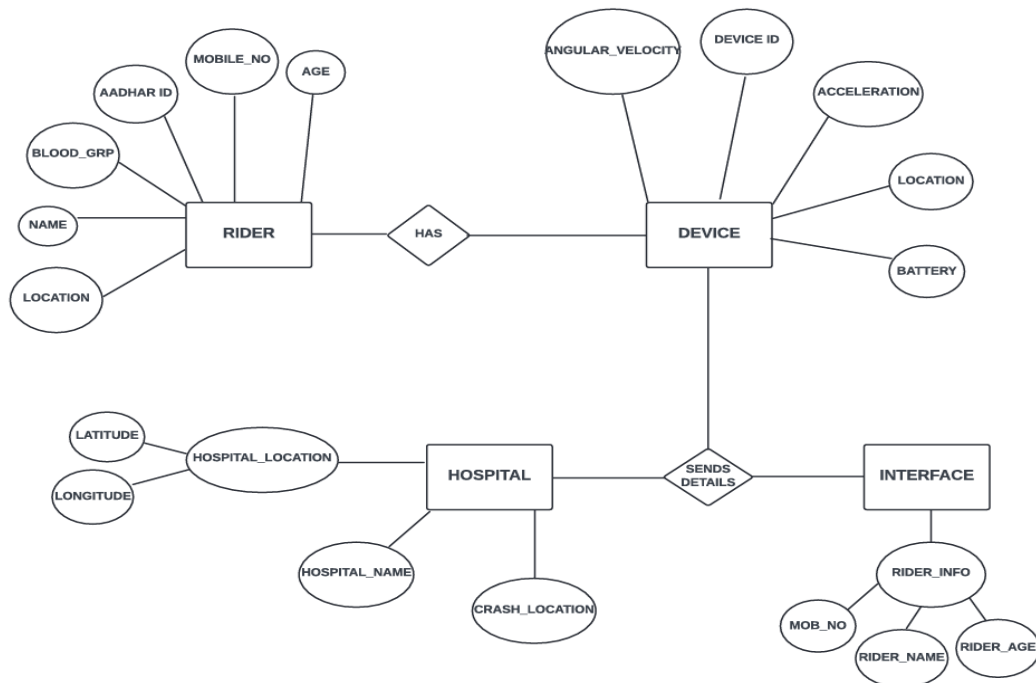


Figure 6: Entity Relationship for Smart Helmet

4.2.3 Class Design Diagram

The smart helmet system involves several key components and their relationships, supporting features like crash detection, emergency alerts, and user management. The User manages login and profile settings, while the Rider tracks details like location, and crash status. The Hospital handles emergency services and patient registration. The Website facilitates functions like creating profiles, detecting nearby hospitals, and displaying routes and crashes detected. The system is interconnected—Users are linked to Riders, Riders to Hospitals, and the Website interacts with multiple components to ensure seamless operation and safety.

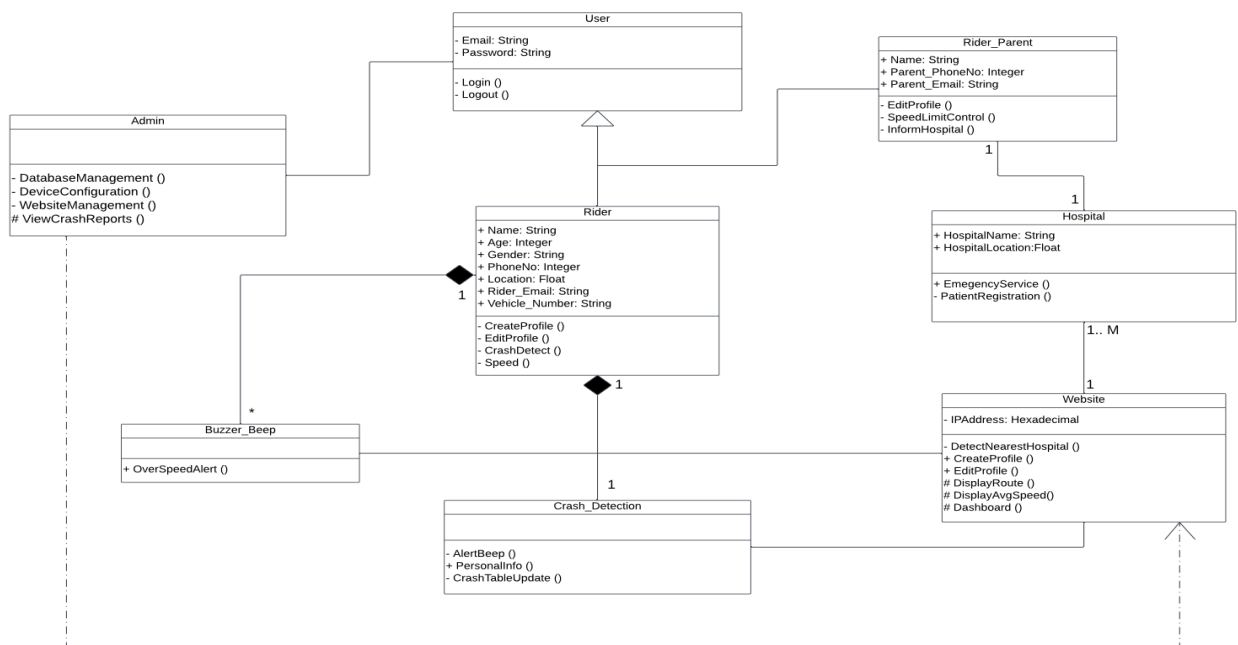


Figure 7: Class Diagram for Smart Helmet

4.3 User Interface Diagrams

4.3.1 Use Case Diagram

The smart helmet system's use case interactions define the roles and functionalities available to different users, guiding interface design and feature development. The User can access most features except for managing device

setup, the database, and the cloud. The Admin manages device setup, the database, cloud operations, emergency services, and user details. This structure clarifies system behaviour and user interactions, ensuring a comprehensive and user-centric design.

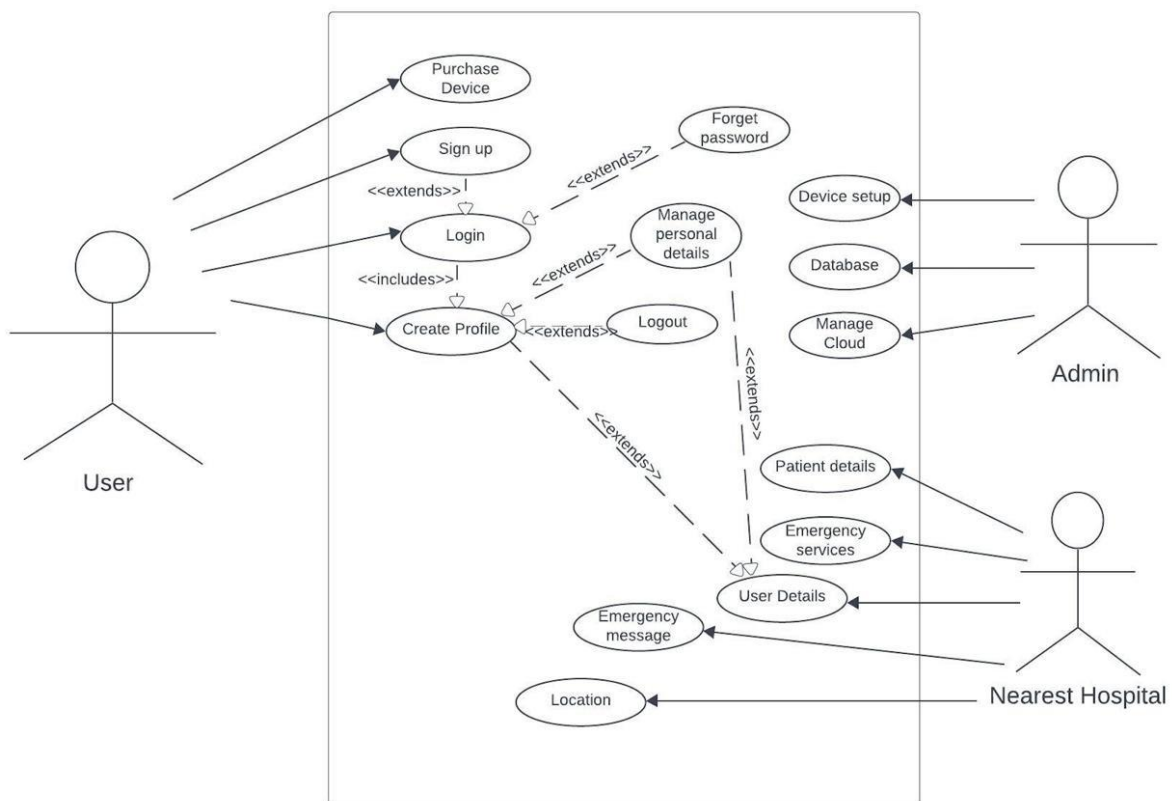


Figure 8: Use Case Diagram for Smart Helmet

4.3.2 Activity Diagram

Key activities and decisions:

- Start: The system begins its operation.
- Crash detected: The system detects a crash based on sensor data.

- Beeps: The system emits beeps to alert the rider.
- Rider Confirms: The rider confirms whether there is a crash or not.
- End: The system ends its operation.

Additional elements:

- IoT Device on helmet: The helmet is equipped with IoT devices to collect data.
- Sensors: The IoT devices use sensors to measure tilt, location (from smartphone), and other parameters.
- Database: The collected data is stored in a database.
- Website: A website is used to display information to users.
- Hospital Interface: The hospitals will have web interface to receive crash details.
- Location Table, Crash Table: These tables store data related to location and crashes.
- Display: The system displays relevant information to the user.

Flow of the system:

1. The system starts.
2. It continuously monitors sensor data to detect a crash.
3. If a crash is detected, alarm beeps for the confirmation whether a crash is happened or not.
4. The rider confirms whether there is a crash or not.
5. If the rider confirms a crash, the system calculates the nearest medical facility and displays the crash alert with relevant information.
6. The system updates its location data.
7. The system ends its operation.

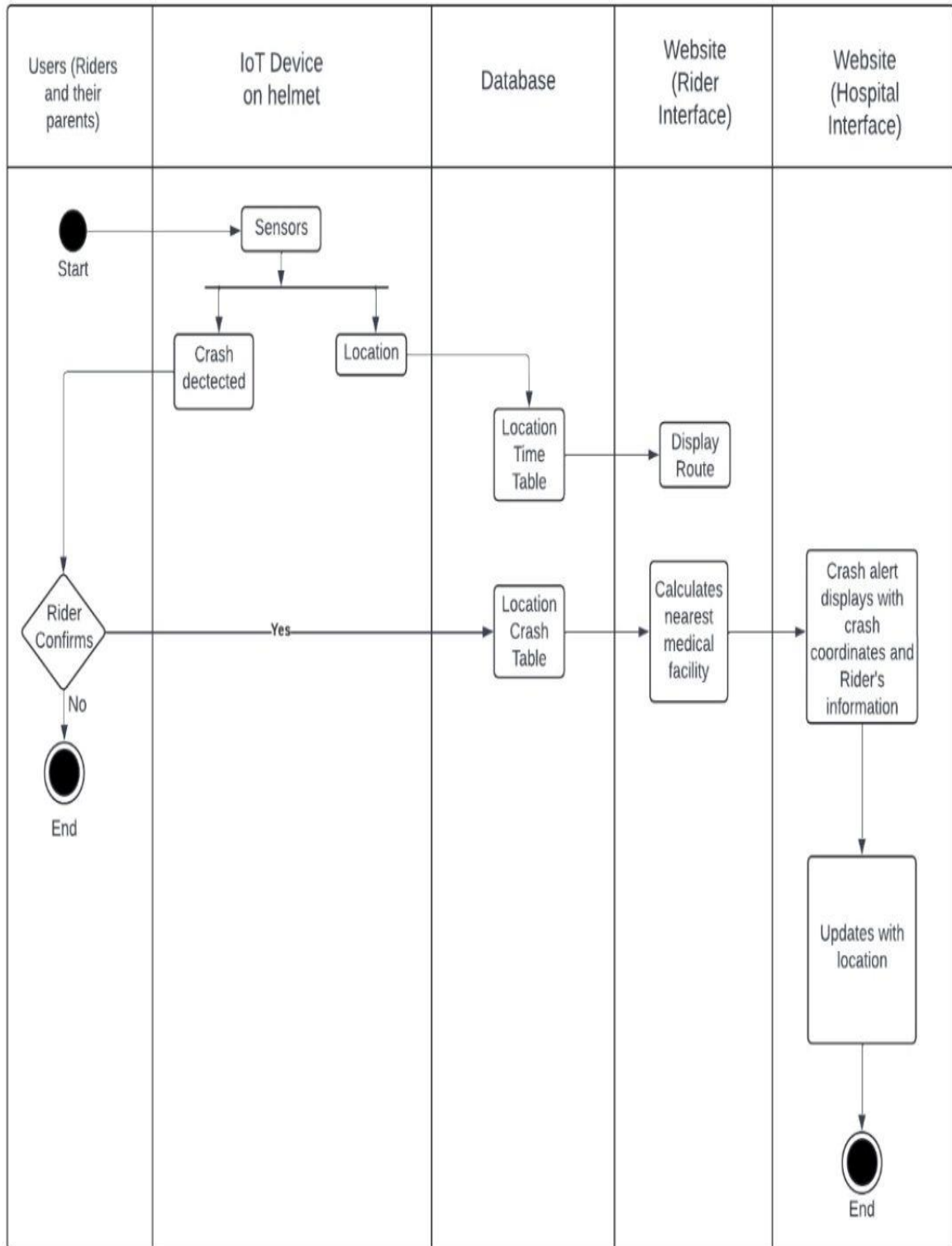


Figure 9: Activity Diagram for Smart Helmet

4.4 Diagram of Prototype

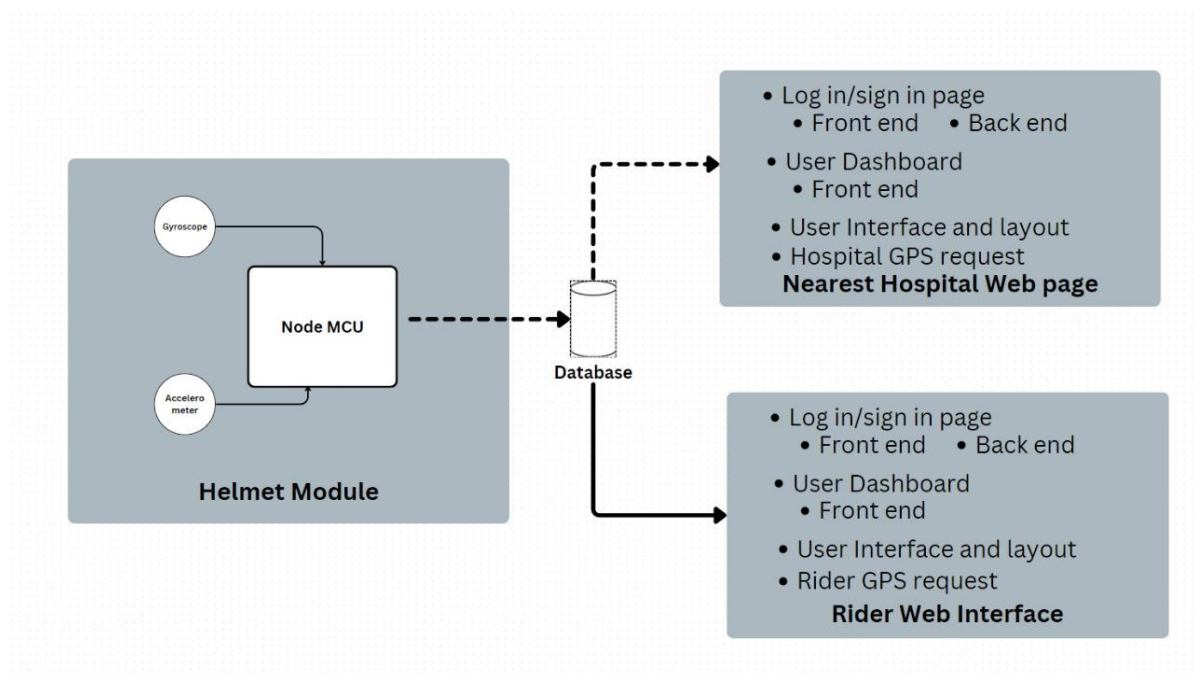


Figure 10: Diagram of Prototype

IMPLEMENTATION AND EXPERIMENTAL RESULT

5.1 Experimental Setup

The experimental setup for the IoT Smart Helmet project demonstrates the implementation and validation of its core functionalities, ensuring the system performs as designed under various scenarios.

Hardware Setup:

1. Helmet Module Components:

- **NodeMCU Microcontroller:** Serves as the central processing unit for data acquisition and communication.
- **MPU6050 Accelerometer and Gyroscope:** Detects crashes based on motion and orientation changes.
- **Buzzer:** Emits a beeping sound upon crash detection to alert nearby individuals.
- **GPS Module:** Captures location data to identify crash sites (uses smartphone GPS).
- **Power Source:** A portable battery powers the system for extended use.

2. Backend Infrastructure:

- **Mongo DB Database:** Stores rider information, crash data, and real-time location.
- **Web Server:** Facilitates communication between the helmet module, database, and user interface.

Software Implementation:

1. Firmware Development:

- Code written in C++ using the Arduino IDE for NodeMCU.
- Integration of accelerometer, gyroscope, and buzzer functionalities.

2. Web Application:

- Developed using the MERN stack.
- Displays crash details and rider information on a dashboard for medical facilities.

3. Communication Protocols:

- Data from the NodeMCU is sent to the server via HTTP over Wi-Fi.
- Alerts and location data are processed on the server and pushed to registered hospitals.

Testing Environment:

1. Location: Experiments conducted in controlled environments simulating real-world scenarios, such as controlled impacts to test crash detection.
2. Tools:
 - Simulated crash setups to test sensor sensitivity and thresholds.
 - Frontend testing with Selenium to verify web interface functionalities (e.g., login, dashboard display).
 - Postman for backend API testing.

Procedure:

1. System Calibration:
 - Sensors calibrated to define thresholds for crash detection.
 - Fine-tuning of accelerometer and gyroscope readings to differentiate between crashes and false positives.
2. Crash Detection Simulation:
 - Impacts of varying intensities applied to the helmet to test crash recognition accuracy.
 - Successful detections trigger the buzzer and send data to the backend.
3. Data Transmission:
 - Real-time data sent to the Mongo DB server upon crash detection.
 - The web application retrieves and displays data for user and hospital interfaces.
4. Emergency Notification:
 - Automatic identification of the nearest registered hospital based on location data.
 - Transmission of crash details and rider information to the hospital interface.

Performance Validation:

1. Metrics:
 - Accuracy of crash detection.
 - Reliability of data transmission to the server.
 - Response time for notifications reaching the nearest hospital.
2. Test Results:
 - Validation of seamless integration between hardware and software components.

- Successful end-to-end flow from crash detection to hospital notification under test scenarios.

5.2 Experimental Analysis

5.2.1 Data

Data for the IoT Smart Helmet system was obtained from multiple sources, including sensors and user devices. The accelerometer and gyroscope collected real-time data to detect sudden impacts, abnormal tilts, or irregular motion, which could indicate a crash. Location data was retrieved using the rider's phone GPS, ensuring accurate tracking of the rider's position during normal operations and emergencies. Additionally, emergency alert triggers generated critical event data, validating the system's capability to promptly notify predefined contacts and hospitals when necessary. The extracted data was thoroughly processed and analysed to ensure the reliability of crash detection and location tracking. The decision-making logic of the system was tested under various crash scenarios to confirm its effectiveness in triggering alerts promptly.

5.2.2 Performance Parameters

The performance of the IoT Smart Helmet system was evaluated based on three key parameters:

Accuracy focused on two critical areas: the percentage of correctly detected crash events using sensor data (accelerometer and gyroscope) and the precision of location tracking achieved through the rider's phone GPS. This ensured the system could provide reliable crash detection and real-time location updates.

Latency measured the time taken to send emergency alerts and update location data to the hospital interface after crash detection. It also considered the time lag between identifying a crash and delivering notifications to predefined emergency contacts and hospitals. Minimizing latency was crucial to ensuring timely emergency responses.

User Experience was assessed based on the usability and intuitiveness of the web interface for both riders and hospital personnel. Feedback was collected to evaluate the clarity, responsiveness, and overall performance of the location tracking dashboard and emergency alert display. Ensuring a smooth and user-friendly

experience for all stakeholders was a priority to maximize the system's effectiveness.

5.3 Working of the project

5.3.1 Procedural Workflow

The system features a well-defined workflow that supports the timely collection and processing of data from the Helmet module to detect any crash accidents. Upon detecting a crash, the system sends the rider's precise location and details to the nearest hospital. This workflow can be categorised into following phases:

1. System Design and Planning Phase

- **Planning:** Define the core objectives for the model. Automatic crash detection and location transmission. Triggering of a beeping sound upon crash detection. Development of a web interface for hospital notifications and rider data.
- **Select and procure hardware components:** To implement the system, we procure hardware components such as a NodeMCU (ESP8266 or ESP32) microcontroller for handling connectivity and data processing, an MPU6050 sensor for capturing accelerometer and gyroscope data to detect motion and impacts, and a buzzer for providing audible alerts in case of crash detection or warnings.
- **System architecture:** Data flow from sensors → NodeMCU → Backend → Web Interface.

2. Hardware Development Phase

- Integrate the MPU6050 accelerometer and gyroscope with the NodeMCU to detect motion, orientation, and crashes, processing real-time data to identify unusual patterns.
- Add a buzzer module to provide audible alerts during critical situations, and configure smartphone GPS integration to obtain and share precise location data for emergency assistance.

- Power the system with a rechargeable battery to ensure uninterrupted operation during rides.

3. Backend Development Phase

- **MongoDB database:** The MongoDB database is designed with four key collections to store and manage the data required for the system's functionality. These collections ensure efficient storage, retrieval, and processing of critical information. The **users** collection stores detailed information about each rider, including their name, contact number, address, emergency contact details, and other personal information. The **datareadings** collection is used for logging sensor data captured by the system. This includes real-time motion data, crash detection flags, and precise location coordinates (latitude and longitude) obtained during operation. The **hospitals** collection maintains records of registered hospitals, including their names, contact details, and location coordinates. This data is used to identify and notify the nearest medical facility in the event of a crash. Finally, the **crasheds** collection stores information about confirmed crash events. It includes the rider's details, the crash time, and the associated hospital to which the alert was sent. This collection serves as a historical record for managing and analysing emergency responses.
- **Develop API:** The system's backend is developed using Node.js and Express.js to handle communication between the IoT device and the database. A set of robust APIs is implemented to manage the flow of data efficiently and ensure seamless functionality. One of the key APIs is designed to receive crash data from the NodeMCU. This API processes sensor data sent by the device, including crash detection flags and location coordinates, and stores the information in the MongoDB database. Another critical API is responsible for sending crash notifications to the nearest registered hospital. To facilitate these operations, the system ensures real-time data transmission using HTTP requests over the Wi-Fi module integrated into the NodeMCU. This setup guarantees prompt and reliable communication between the IoT device and the backend.

4. Web Interface Development Phase

- **User Dashboard:** A web-based dashboard is developed using React.js to provide an intuitive and user-friendly interface for managing and displaying critical information. This dashboard serves as the primary interface for accessing and visualizing system data. The dashboard displays rider information, including personal details such as name, contact number, and emergency contact information. Additionally, it provides the crash location coordinates, allowing users to view the precise site of the crash, often visualized on a map for better clarity and situational awareness.
- **Hospital Dashboard:** A dedicated hospital-specific web interface is also developed to streamline communication with medical facilities. This interface is designed to receive crash alerts and the associated location details, enabling hospitals to respond quickly to emergencies.

5. System Integration Phase

- The system's hardware, backend, and web interface are seamlessly integrated to ensure smooth functionality and efficient communication across all components. This integration enables the system to operate as a unified solution for crash detection and response. Communication between the NodeMCU and the backend is established to enable the transmission of crash alerts. The NodeMCU sends real-time sensor data, including crash detection flags and location information, to the backend over a Wi-Fi connection.
- The backend is verified to ensure that it properly updates the MongoDB database with sensor data received from the NodeMCU. This ensures that all crash-related data is accurately stored for further processing and retrieval.
- The web interface is configured to retrieve data from the backend and display it in real-time. This allows users and hospitals to access the most recent crash details and rider information immediately, ensuring timely decision-making.

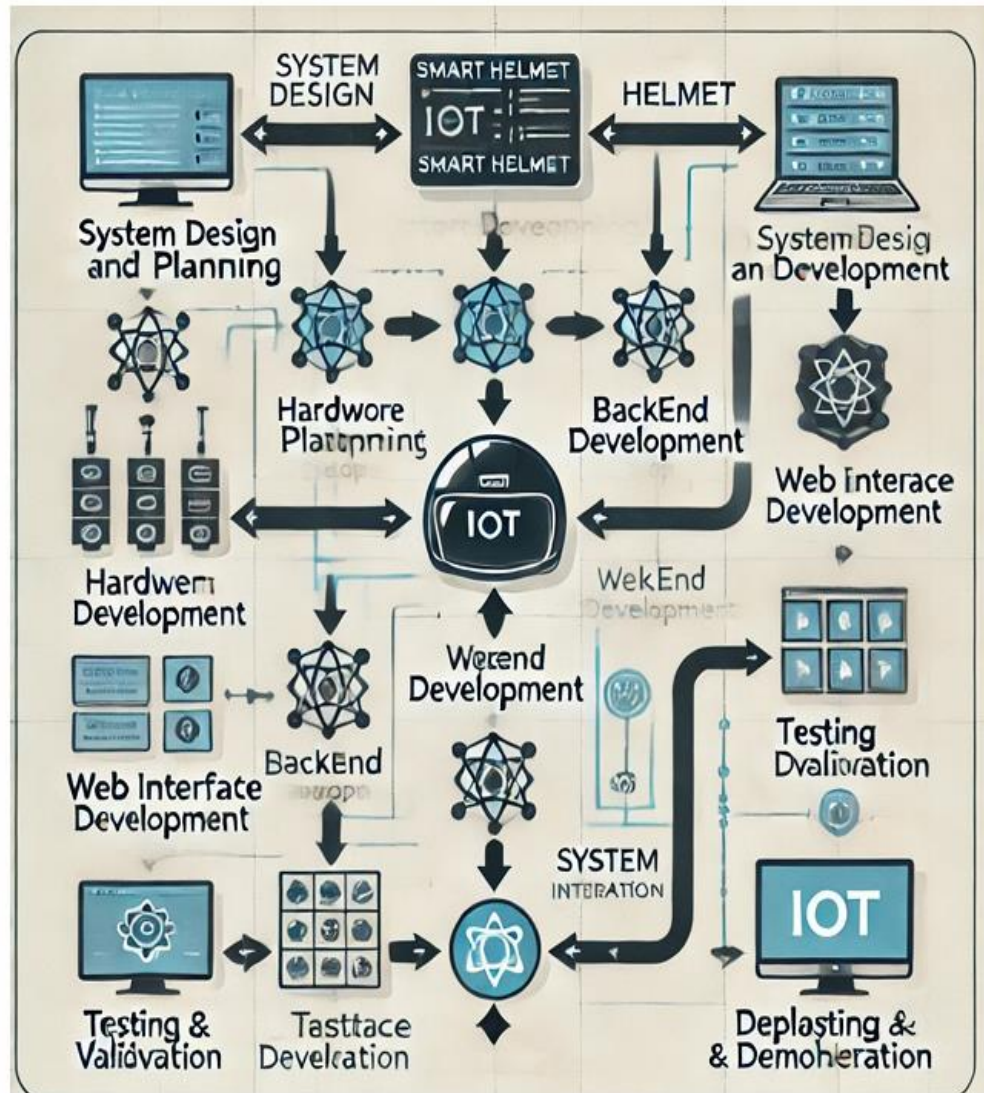


Figure 11: Procedural Workflow

5.3.2 Algorithmic Approaches Used

1. Hardware Pseudo code:

a. Global Declarations:

1. Define WiFi credentials:

- SSID: "xyz"
- Password: "abc"

2. Define thresholds for acceleration:

- ACCEL_X_THRESHOLD = 1.25
- ACCEL_Y_THRESHOLD = 0.40

- ACCEL_Z_THRESHOLD = 0.48
3. Define pin connections:
 - buzzerPin = Pin 14 (D5)
 4. Create a data structure MyData to store:
 - Device ID
 - Mapped sensor values (X, Y, Z)
 - Acceleration values (accelX, accelY, accelZ)
 5. Initialize CrashConfirmation to false.

b. Setup Function:

1. Start Serial communication at 9600 baud rate.
2. Connect to WiFi network:
 - Attempt to connect.
 - Print connection status and IP address upon success.
3. Initialize MPU6050 sensor:
 - Set up I2C communication (SDA = GPIO4, SCL = GPIO5).
 - Check if MPU6050 is connected. If not, stop execution.
4. Set buzzerPin as OUTPUT and turn it off initially.

c. Helper Functions:

1. isAccelerationExceedingThresholds Function:
 - Check if the absolute acceleration values for X, Y, or Z exceed their respective thresholds.
 - Return true if any threshold is exceeded, otherwise false.
2. updateSensorReadings Function:
 - Fetch acceleration (ax, ay, az) and gyroscope data from MPU6050.
 - Map acceleration values (ax, ay, az) to a range of 0 to 255 and store as X, Y, Z.
 - Convert raw acceleration data to g values and store in accelX, accelY, accelZ.
3. printSensorData Function:

- Print tilt values (X, Y, Z).
- Print acceleration values (accelX, accelY, accelZ) in g.

4. sendCrashReport Function:

- Check if WiFi is connected.
- Create an HTTP POST request to the server endpoint (http://16.16.184.223:4000/api/readings).
- Include JSON body with:
 - crashDetected = true
- Add necessary headers, including authorization.
- Send the request and print server response or error message.
- Close the HTTP connection.

d. Main Loop Function:

1. Read Sensor Data:

- Call updateSensorReadings.
- Print sensor data using printSensorData.

2. Check for Crash Detection:

- If acceleration exceeds thresholds (isAccelerationExceedingThresholds):
 - Activate buzzer for 10 iterations:
 - Turn buzzer ON for 1 second.
 - Turn buzzer OFF for 1 second.
 - Update sensor readings during each iteration.
 - If thresholds are no longer exceeded:
 - Print "False Crash" and reset CrashConfirmation to false.
 - Exit the loop.
- If thresholds remain exceeded after 10 iterations:
 - Set CrashConfirmation to true.
 - Print "Crash Confirmed".
 - Call sendCrashReport to notify the server.

3. Delay for Stability:

- Add a delay of 2 seconds before the next iteration.

Explanation:

1. Setup Phase:

- Initializes WiFi connection for communication.
- Sets up the MPU6050 sensor for reading motion data.
- Configures the buzzer pin to act as an alert indicator.

2. Sensor Reading:

- Continuously fetches acceleration values (X, Y, Z axes) from the MPU6050 sensor.
- Converts the raw sensor data into readable tilt and acceleration values.

3. Crash Detection Logic:

- Checks if any of the acceleration values exceed predefined thresholds.
- If thresholds are exceeded, the buzzer is activated for 10 iterations.
- During this period, sensor values are monitored to verify if the event is a true crash or a false trigger.
- If the thresholds remain exceeded after 10 checks, the crash is confirmed.

4. Crash Reporting:

- When a crash is confirmed, a POST request is sent to a server endpoint with a JSON payload indicating `crashDetected: true`.
- The response from the server is logged for debugging.

5. Loop Execution:

- The process of reading sensor data, checking thresholds, and verifying crashes repeats every 2 seconds to ensure continuous monitoring.

2. Data Transmission upon Crash Algorithm

Once a crash is detected, the system transmits crash data, including rider details and location coordinates, to the backend server using HTTP POST requests over Wi-Fi.

Pseudo code:

Retrieve the token from cookies (hosuser).

Fetch crash data from the endpoint /api/crashes with authorization:

- Parse and store crash data into crashData.
- Extract latitude, longitude, and email from the latest reading of each crash.
- Update the coordinate's state.

Fetch hospital location from /api/hospitals with authorization:

- Parse and store hospital latitude and longitude in hospitalLocation.

On failure:

- Log the error and show a toast notification.

Set loading to false at the end.

Explanation:

Token Retrieval: The function retrieves an authentication token (hosuser) stored in cookies to authorize API requests.

Fetch Crash Data:

- Sends a GET request to /api/crashes with the token.
- On success, it updates the state crashData with the received data.
- Extracts user locations (latitude, longitude, and email) from the latestReading field of the crash data and updates the coordinate's state.

Fetch Hospital Location:

- Sends another GET request to /api/hospitals with the same token.
- Stores the hospital's location (latitude and longitude) in the hospitalLocation state.

Error Handling: If the requests fail, an error is logged to the console, and a notification is displayed using toast.error().

Loading State: At the end of the process (regardless of success or failure), the loading state is set to false to indicate that the data-fetching process has been completed.

3. Nearest Hospital Selection Algorithm

1. createDataReading Function

Input: req (request object), res (response object)

Purpose: Create a data reading entry and detect crashes to find the nearest hospitals.

Pseudo code:

1. Initialize a new DataReading object with:
 - userId (from the authenticated user).
 - Other request body data (req.body).
2. If crash is detected (req.body.crashDetected is true):
 - Find the latest reading for the user where coordinates are not empty.
 - If a latest reading exists:
 - a. Extract coordinates from the location (latitude, longitude).
 - b. Use GeoSpatial Query to find the nearest hospitals within a 5km radius:
 - Ensure coordinates are not null in the hospital data.
 - c. For each nearest hospital, create an entry in the Crashed table with:
 - hospitalId
 - userId
 - Current timestamp (time).
 - d. Insert all crash entries into the Crashed table.
3. Save the newly created DataReading object to the database.
4. Return a 201 status with the saved data.
5. Error Handling: Return a 400 status with the error message on failure.

2. getDataReadings Function

Input: req, res

Purpose: Retrieve all data readings for the logged-in user.

Pseudo code:

1. Find all DataReading records where userId matches the authenticated user (req.user.id).
2. Return a 200 status with the list of readings.
3. Error Handling: Return a 500 status with the error message on failure.

3. Protect Middleware

Input: req, res, next

Purpose: Authenticate the user using JWT and set the user in the request object.

Pseudo code:

1. Check if the Authorization header exists and starts with Bearer.
2. If no header, return a 401 status with a "Not authorized" message.
3. Extract the token from the header.
4. Use `getUserFromToken` to decode the token and retrieve the user:
 - If no user is found, return a 401 status.
5. Attach the user object to `req.user`.
6. Call `next()` to proceed to the next middleware or route handler.
7. Error Handling: Return a 401 status with a "Not authorized" message on failure.

4. `getLatestDataReading` Function

Input: `req, res`

Purpose: Retrieve the most recent data reading for the logged-in user.

Pseudo code:

1. Find the latest `DataReading` entry for the user (sorted by `createdAt` in descending order).
2. If no reading is found, return a 404 status with "No readings found".
3. Return a 200 status with the latest reading.
4. Error Handling: Return a 500 status with the error message on failure.

Explanation:

1. `createDataReading`: Creates a new data entry, detects crashes, finds nearby hospitals using geospatial queries, and records the crash details.
2. `getDataReadings`: Retrieves all data readings for the authenticated user.
3. `protect`: Middleware that verifies the user's JWT token to ensure they are authorized.
4. `getLatestDataReading`: Fetches the most recent data reading for the user.

5.3.3 Project Deployment

The deployment of the IoT Smart Helmet system involves integrating hardware components (sensors, NodeMCU, buzzer, GPS module) with a backend infrastructure (database). Real-time data flow between components ensures crash detection, emergency alerts, and effective communication with hospitals or

emergency responders. The component diagram visually represents these interactions, showcasing the modular and scalable nature of the system. Each component operates independently yet collaboratively, ensuring a reliable and efficient crash detection and alerting system.

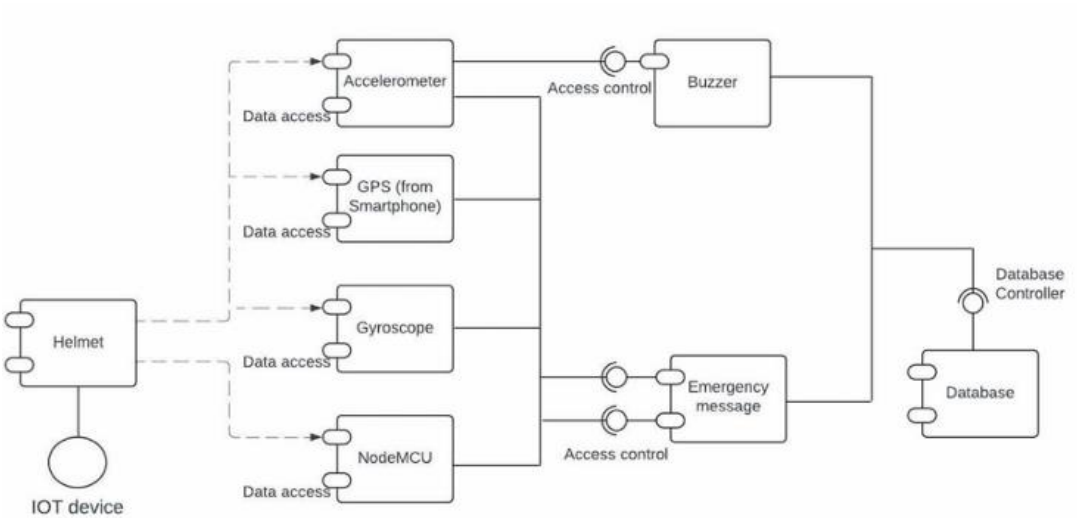


Figure 12: Component Diagram for Smart Helmet

5.3.4 System Screenshots

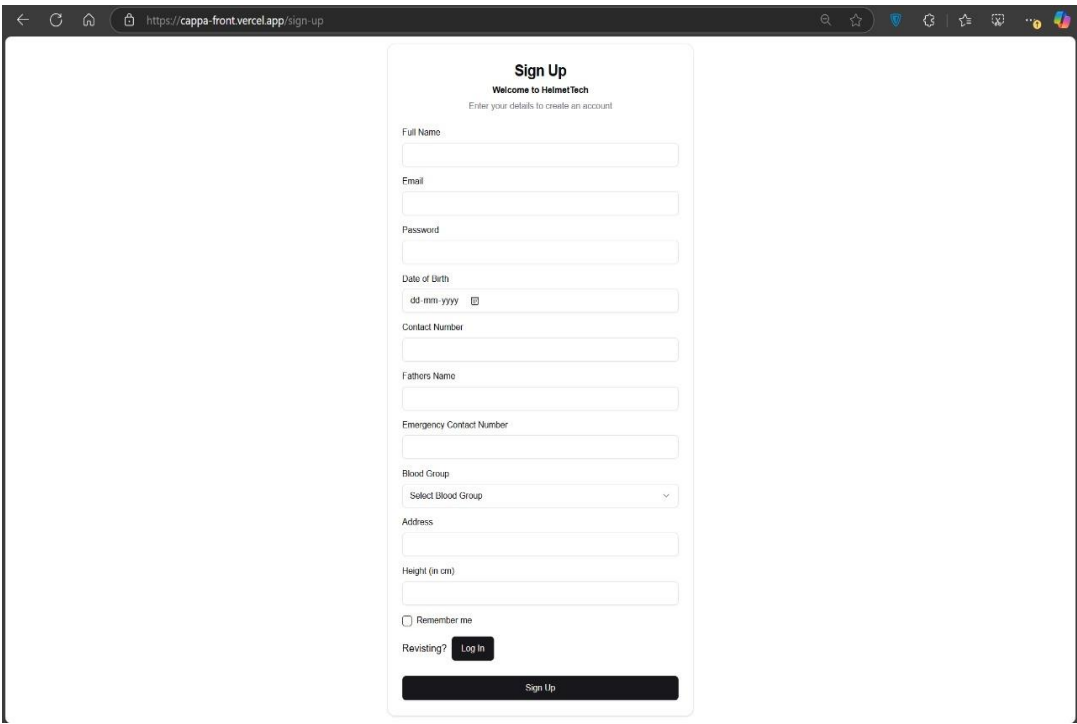


Figure 13: User Signup

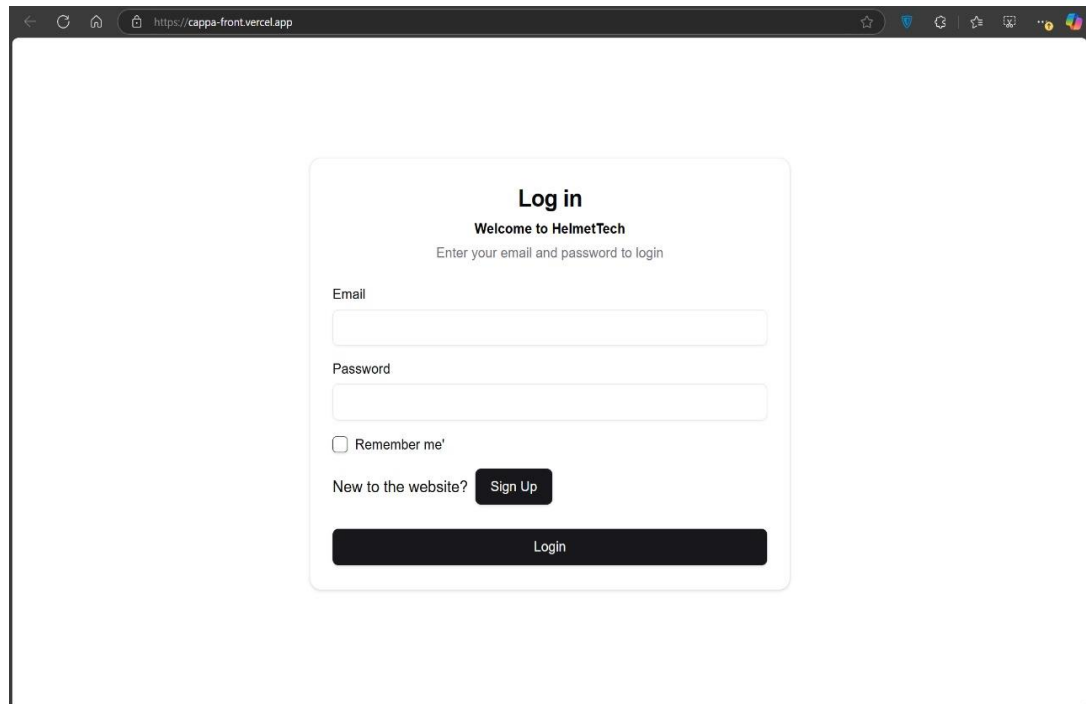


Figure 14: User Login

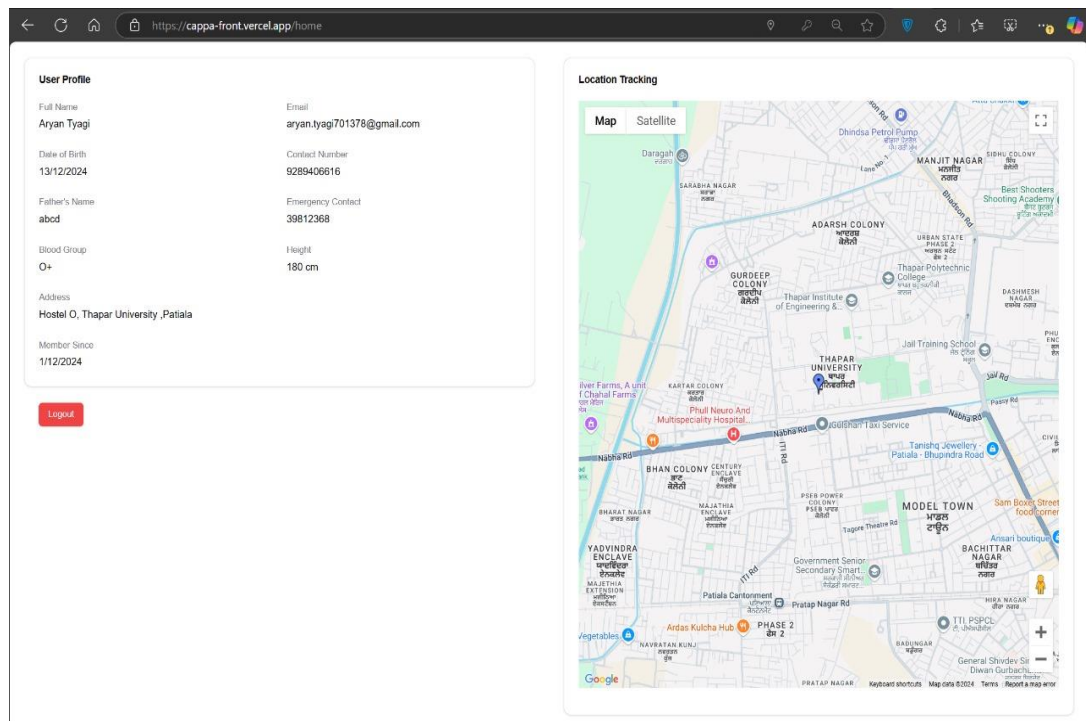
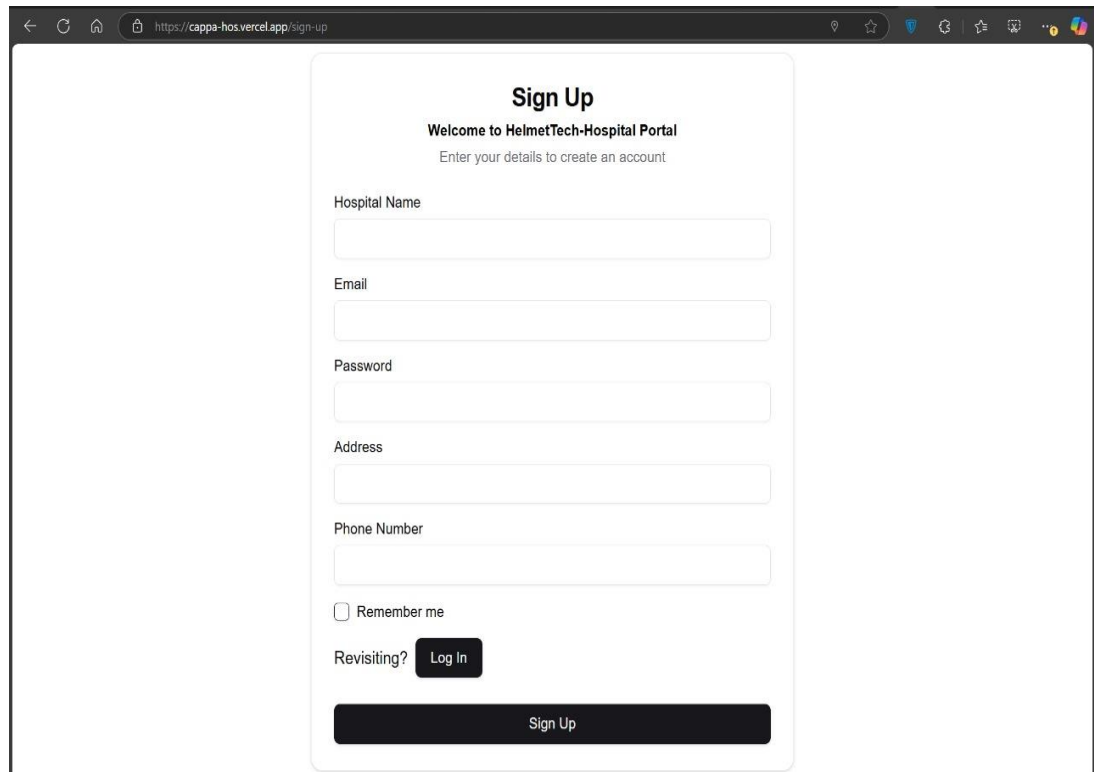


Figure 15: User home page with GPS tracking



A screenshot of a web browser displaying the 'Sign Up' page for the 'HelmetTech-Hospital Portal'. The browser's address bar shows 'https://cappa-hos.vercel.app/sign-up'. The page features a central white card with a light gray border. At the top of the card, the title 'Sign Up' is in bold, followed by the subtitle 'Welcome to HelmetTech-Hospital Portal' and the instruction 'Enter your details to create an account'. Below this, there are six input fields: 'Hospital Name', 'Email', 'Password', 'Address', 'Phone Number', and a 'Remember me' checkbox. At the bottom of the card, there is a 'Revisiting?' label, a 'Log In' button, and a large 'Sign Up' button.

Sign Up

Welcome to HelmetTech-Hospital Portal

Enter your details to create an account

Hospital Name

Email

Password

Address

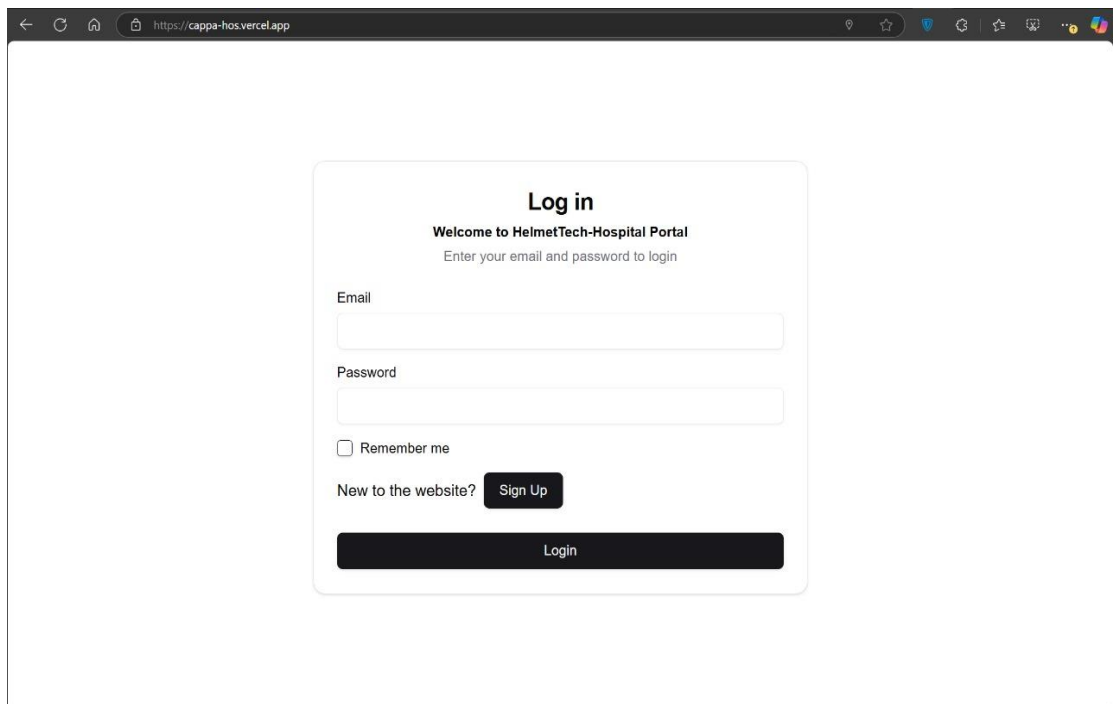
Phone Number

☐ Remember me

Revisiting? [Log In](#)

[Sign Up](#)

Figure 16: Hospital Signup



A screenshot of a web browser displaying the 'Log in' page for the 'HelmetTech-Hospital Portal'. The browser's address bar shows 'https://cappa-hos.vercel.app'. The page features a central white card with a light gray border. At the top of the card, the title 'Log in' is in bold, followed by the subtitle 'Welcome to HelmetTech-Hospital Portal' and the instruction 'Enter your email and password to login'. Below this, there are two input fields: 'Email' and 'Password'. At the bottom of the card, there is a 'Remember me' checkbox, a 'New to the website?' label, a 'Sign Up' button, and a large 'Login' button.

Log in

Welcome to HelmetTech-Hospital Portal

Enter your email and password to login

Email

Password

☐ Remember me

New to the website? [Sign Up](#)

[Login](#)

Figure 17: Hospital Login

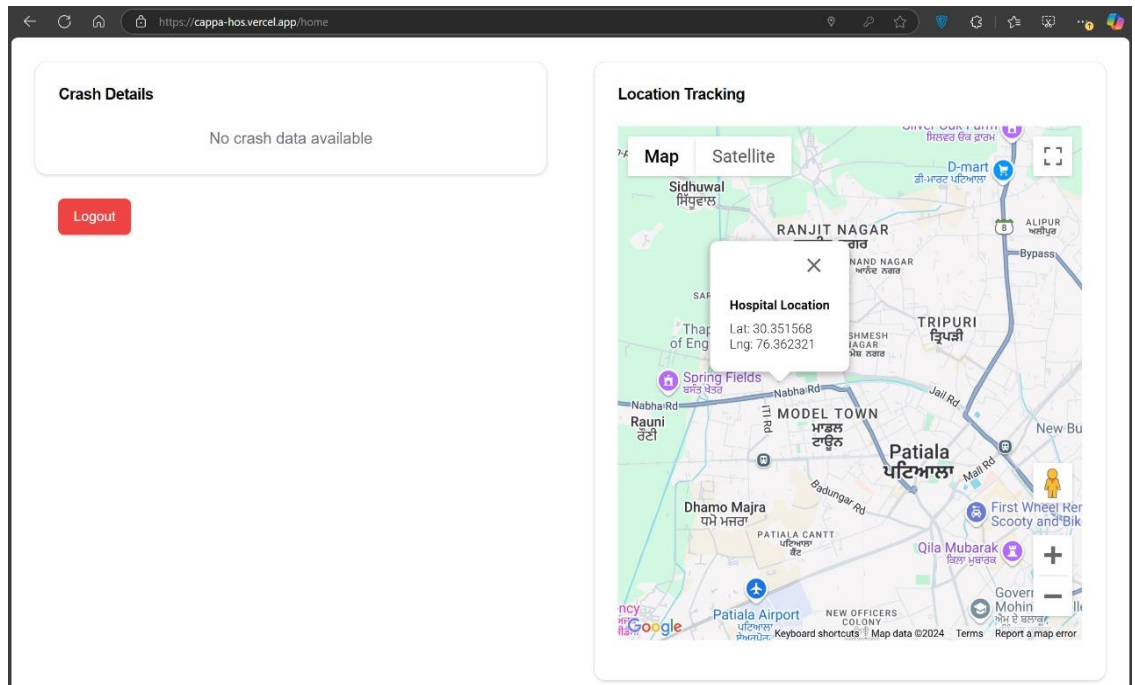


Figure 18: Hospital homepage

5.4 Testing Process

5.4.1 Test Plan

The purpose of this test plan is to define the scope, objectives, and methodology for testing the Smart Crash Detection System's website. The website plays a critical role in tracking riders, ensuring proper authentication, and providing details related to crash detection. This plan aims to ensure that core functionalities such as login, signup, and logout work seamlessly under various scenarios.

Scope of Testing

- Validate the authentication module (login, signup, and logout).
- Verify smooth navigation between pages (login/signup → home → logout).
- Ensure system behaviour under invalid inputs to maintain security and usability.
- Test user interface (UI) and experience (UX) to ensure clarity and responsiveness.

Objectives

- Detect any defects in user interactions with the authentication system.
- Confirm the reliability of redirection flows between pages (login/signup → home page).
- Ensure that the logout button effectively clears session data and redirects appropriately.
- Validate error-handling mechanisms (e.g., error messages for invalid credentials).

Testing Environment

- Frontend Technology: React.js.
- Backend Hosting: Vercel for APIs and static files.
- Browser: Google Chrome.
- Automation Tool: Selenium Web Driver with J Unit for testing.
- Test Data: Predefined credentials and dummy user details.

Assumptions and Risks

- The system is expected to handle both valid and invalid inputs without crashing.
- Risks include potential API failures during authentication or incorrect form validation.

5.4.2 Features to be tested

1. Login with Wrong Credentials:

Objective: Ensure the system denies access to users with invalid credentials.

Test Scenarios:

- Enter a valid email but an incorrect password.
- Enter a non-existent email with any password.
- Leave the email and password fields empty and attempt to log in.

Expected Results:

- An appropriate error message, such as "Invalid email or password," should be displayed.
- No redirection to the home page should occur.

2. Login with Correct Credentials:

Objective: Validate that users with valid credentials can access the home page.

Test Scenarios:

- Enter a registered email and the corresponding password.
- Ensure the home page displays correct rider details upon successful login.

Expected Results:

- Successful login should redirect to the home page.
- The user should remain logged in unless they log out or their session expires.

3. Signup Functionality:

Objective: Verify that new users can register successfully and access the system.

Test Scenarios:

- Enter valid inputs (hospital name, email, password, address, and phone number) and click "Sign Up."
- Attempt signup with invalid or incomplete data (e.g., missing fields, invalid email format).

Expected Results:

- Valid inputs should create an account and redirect the user to the home page.
- Invalid inputs should display appropriate error messages and prevent account creation.

4. Logout Functionality:

Objective: Ensure that users can log out of the system securely.

Test Scenarios:

- Click the "Logout" button after successful login.
- Attempt accessing the home page without logging in after logging out.

Expected Results:

- Clicking the "Logout" button should clear the session and redirect to the login/signup page.
- Accessing the home page without logging in should redirect back to the login page.

5.4.3 Test Strategy

The test strategy focuses on a black-box testing approach to evaluate the functionality of the authentication module without delving into the internal code.

1. Test Types:

- Functional Testing: Ensures the login, signup, and logout processes perform as expected.
- Negative Testing: Tests the system's response to invalid or edge-case inputs.
- UITesting: Verifies that the form elements (input fields, buttons) are displayed correctly and are user-friendly.

2. Execution Flow:

- Execute each feature test independently to isolate issues.
- Automate repetitive tasks using Selenium WebDriver.
- Record detailed logs for every test case, including steps, inputs, and outputs.

3. Defect Management:

- Use a defect tracker to log any bugs discovered during testing.
- Prioritize critical issues (e.g., failure to log in) for immediate resolution.

4. Exit Criteria:

- All critical test cases for login, signup, and logout must pass successfully.
- All identified bugs must be resolved or deferred based on their severity.

5.4.4 Test Techniques

To achieve the testing objectives, the following techniques will be employed:

1. Boundary Value Analysis (BVA):

- Test inputs near the boundaries of acceptable ranges (e.g., minimum password length, email format).

2. Equivalence Partitioning (EP):

- Divide input data into valid and invalid equivalence classes (e.g., valid email formats vs. invalid ones).

3. Error Guessing:

- Anticipate potential error scenarios (e.g., leaving fields empty, entering special characters) and test for these cases.

4. End-to-End Testing:

- Simulate complete user flows, such as signing up, logging in, navigating to the home page, and logging out.

5. Automated Testing:

- Use Selenium Web Driver to automate form-filling, button-clicking, and navigation.
- Automate repetitive test cases to save time and ensure consistency.

6. Cross-Browser Testing:

- Test the system on multiple browsers (e.g., Chrome, Firefox) to ensure compatibility.

7. Usability Testing:

- Evaluate the website's usability by observing how intuitive and easy-to-navigate the login/signup forms are for users.

5.4.5 Test Cases

Here's a detailed breakdown of the test cases for each feature mentioned:

Table 5: Login with Wrong Credentials

Test Case ID	Description	Test Steps	Expected Result	Status
TC-01	Empty email and password	1. Navigate to the login page. 2. Leave the email and password fields blank. 3. Click the "Login" button.	Error message: "Email and password are required." No redirection occurs.	Tested
TC-02	Invalid email format	1. Enter an invalid email (e.g., <code>testemail</code>). 2. Enter any password. 3. Click the "Login" button.	Error message: "Invalid email format." No redirection occurs.	Tested
TC-03	Incorrect email/password	1. Enter a non-existent email or correct email with an incorrect password. 2. Click the "Login" button.	Error message: "Invalid email or password." No redirection occurs.	Tested

Table 6: Login with Correct Credentials

Test Case ID	Description	Test Steps	Expected Result	Status
TC-04	Successful login	1. Enter valid email and password. 2. Click the "Login" button.	User is redirected to the home page. Rider details are displayed.	Tested
TC-05	Session persistence	1. Log in successfully. 2. Refresh the home page.	User remains logged in (session persists).	Tested
TC-06	Failed login after logout	1. Log in successfully. 2. Click the "Logout" button. 3. Try accessing the home page without logging in again.	User is redirected to the login page.	Tested

Table 7: Signup functionality

Test Case ID	Description	Test Steps	Expected Result	Status
TC-07	Valid signup	1. Navigate to the signup page. 2. Enter valid hospital name, email, password, address, and phone number. 3. Click the "Sign Up" button.	Account is created successfully. User is redirected to the home page.	Tested
TC-08	Missing required fields	1. Leave one or more fields blank (e.g., email, password). 2. Click the "Sign Up" button.	Error message: "All fields are required." Account is not created.	Tested
TC-09	Invalid email format	1. Enter an invalid email format (e.g., hospitalname). 2. Click the "Sign Up" button.	Error message: "Invalid email format." Account is not created.	Tested
TC-10	Weak password	1. Enter a password shorter than the minimum length (e.g., "123"). 2. Click the "Sign Up" button.	Error message: "Password must be at least X characters long." Account is not created.	Tested
TC-11	Duplicate email	1. Enter an email that already exists in the system. 2. Click the "Sign Up" button.	Error message: "Email already in use." Account is not created.	Tested

Table 8: Logout functionality

Test Case ID	Description	Test Steps	Expected Result	Status
TC-12	Logout redirection	1. Log in successfully. 2. Click the "Logout" button.	User is redirected to the login page.	Tested
TC-13	Access without logging in	1. Log out successfully. 2. Try accessing the home page by entering the URL directly.	User is redirected to the login page.	Tested

5.4.6 Test Results

1. Emailable Report

- **Description:** The emailable report is a simple HTML report generated after the test execution. It provides a summary of the test results in a format that can be easily shared via email.
- **Contents:**
 - Total tests executed, passed, failed, and skipped.
 - Basic details like test name, duration, and status (Pass/Fail).
- **Purpose:**
 - Quickly communicate test outcomes to stakeholders.
 - Suitable for sharing high-level results in environments where quick reporting is required.
- **Advantages:**
 - Lightweight and easy to generate.
 - Can be viewed on any browser without additional tools.



Test	# Passed	# Skipped	# Retried	# Failed	Time (ms)	Included Groups	Excluded Groups
Default suite							
Default test	1	0	0	0	42,975		

Class	Method	Start	Time (ms)
Default suite			
Default test — passed			
newpackage.firstclassTest	fantasy	1714553404924	42924

Default test

newpackage.firstclassTest#fantasy

Figure 19: Emailable Report

2. Index Report

- **Description:** The index report is a more detailed HTML report that provides an overview and in-depth details of test executions. It acts as a centralized dashboard for test results.

- **Contents:**

- Comprehensive test execution status (total, passed, failed, skipped).
- Links to individual test cases with detailed logs, error messages, and stack traces for failures.
- Information on test classes and methods.

- **Purpose:**

- To give testers and developers deeper insights into test execution and help identify issues.
- Suitable for troubleshooting and debugging failed tests.

- **Advantages:**

- Provides detailed logs for failures.
- Interactive links for quick navigation to specific test cases.

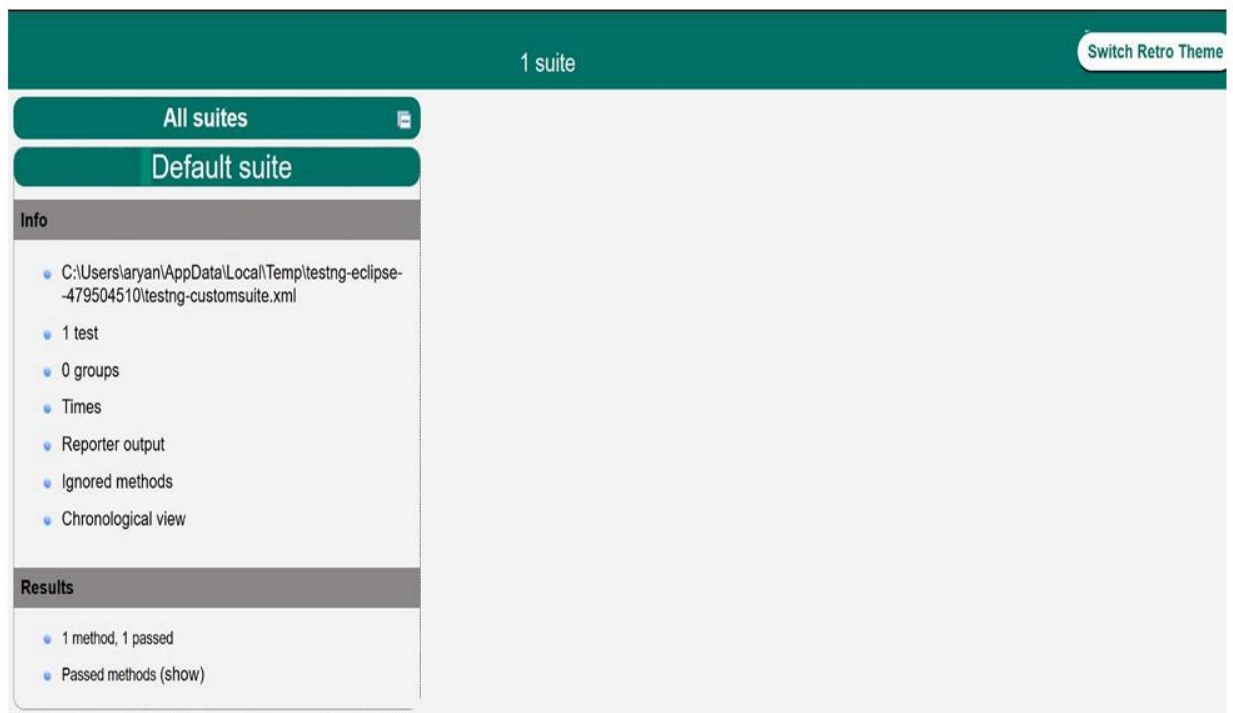


Figure 20: Index Report

3. XSLT Report

- **Description:** The XSLT report (Extensible Stylesheet Language Transformations) is an advanced and visually rich report that uses XSLT to transform raw XML data into a user-friendly HTML report.

- **Contents:**

- Interactive charts and graphs for visualizing test execution data (e.g., pie charts showing pass/fail percentages).
- Detailed information on test cases, errors, and execution times.
- Categorized data for classes, packages, and test suites.

- **Purpose:**

- Provides a polished and visually appealing overview of test results.
- Suitable for presentations to management and stakeholders who prefer graphical representations.

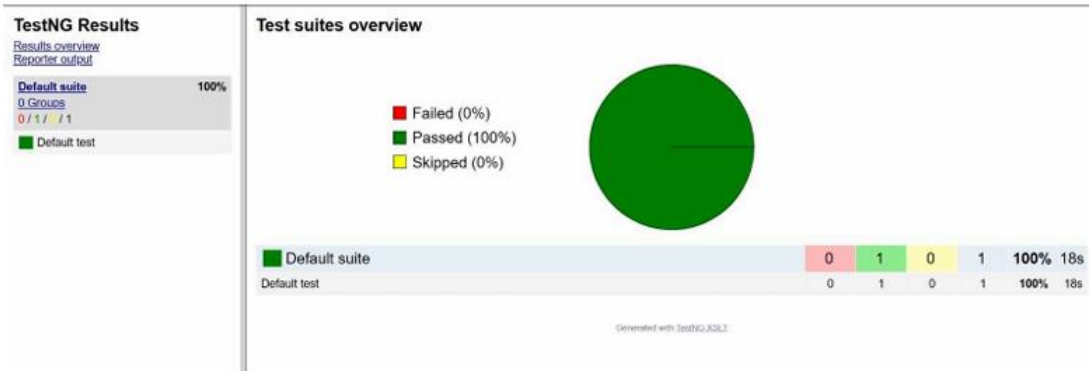
- **Advantages:**

- Offers detailed insights along with visual summaries.
- Highly customizable if you need to adapt the report format.

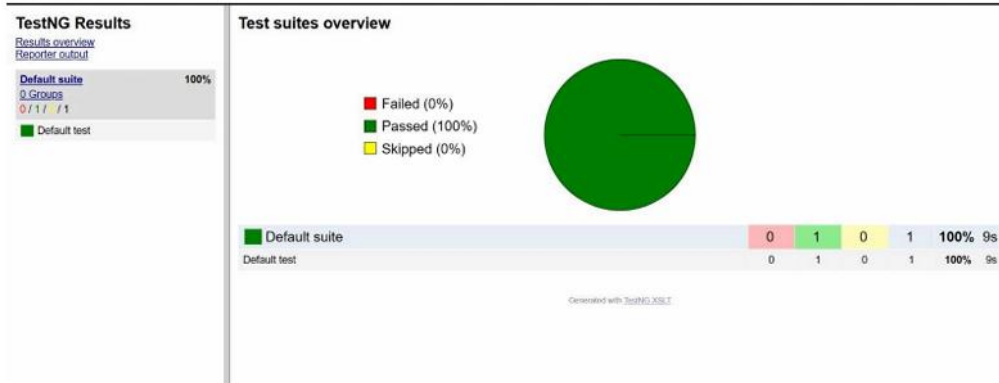
Feature 1:



Feature 2:



Feature 3:



Feature 4:

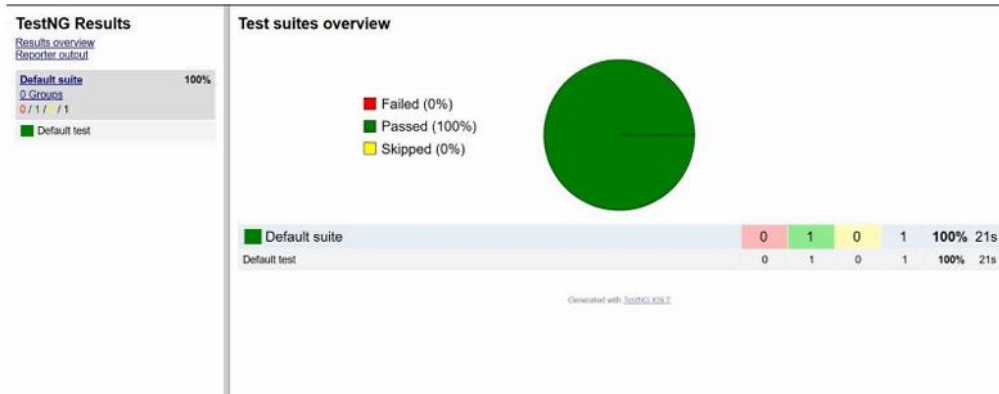


Figure 21: XSLT Report

5.5 Results and Discussions

Visualization of Results using Graph Plots

The results of the IoT Smart Helmet system were analysed and visualized using various graph plots to demonstrate the system's performance across key parameters:

1. Crash Detection Accuracy: A graph was plotted to show the accuracy of crash detection based on data from the accelerometer and gyroscope sensors. The results demonstrated high accuracy, with minimal false positives and false negatives, ensuring the system's reliability under various simulated crash scenarios.

2. Location Tracking Precision: Location accuracy was visualized by comparing the rider's phone GPS coordinates with actual test locations. A scatter plot was used to map the tracked locations, showing that the system consistently provided precise location updates within an acceptable error margin.

3. Latency Analysis: A line graph was used to analyse the latency between crash detection and emergency alert delivery. Results showed that the time lag remained minimal, with notifications sent within a few seconds of a detected crash event, enabling prompt emergency responses.

Comparison with Related State-of-the-Art Work

- **Crash Detection:** Unlike traditional systems that rely solely on tilt sensors, the IoT Smart Helmet integrates both accelerometer and gyroscope data, enhancing the accuracy of crash detection. Compared to existing systems, our solution reduces false triggers by accounting for multiple sensor readings to confirm crash events.
- **Location Tracking:** Most state-of-the-art helmet solutions use dedicated GPS modules, which can add cost and complexity. In contrast, our system leverages the rider's phone GPS, ensuring both cost efficiency and widespread compatibility without compromising tracking precision.
- **Latency in Emergency Alerts:** Compared to other systems, where notification delays may occur due to multiple processing steps, the IoT Smart Helmet achieves low latency by directly transmitting crash data and GPS location to predefined contacts and hospitals using efficient server communication.

5.6 Inferences Drawn

Based on the full implementation and experimental results of IoT based Smart Helmet system, several inferences can be drawn from them:

1. **Sensor Performance:** Calibration of accelerometers and gyroscopes is critical to accurately distinguish between real crashes and false positives caused by sudden braking.
2. **Data Transmission:** Reliable communication using modules is challenging in areas with poor network coverage, requiring optimization and backup mechanisms for uninterrupted emergency alerts.
3. **GPS Limitations:** GPS modules often fail to provide accurate location data indoors or under roofs, emphasizing the need for alternate geolocation techniques like using the mobiles network connectivity.
4. **Power Management:** High power consumption by NodeMCU and communication modules can limit battery life, necessitating power-saving strategies or hardware upgrades.
5. **User Experience:** A responsive and intuitive dashboard, built using Next.js, enhances usability for monitoring real-time data and emergency updates, making it a critical component for user satisfaction.

These inferences guided iterative improvements and ensured the development of a robust and reliable system.

5.7 Validation of Objectives

The objectives of the Smart Parking System, as outlined in the project scope, were successfully validated:

Objective 1: Automatically send precise coordinates and rider details to nearby medical facilities upon detecting a crash

- Status : Fully Achieved
- Validation:

- The MPU6050 accelerometer and gyroscope sensor was integrated and calibrated to detect crashes based on motion and orientation changes exceeding predefined thresholds.
- Upon crash detection, the system successfully transmitted rider details (name, contact information) and location coordinates to the backend, powered by Mongo DB.
- A server-side algorithm identified the nearest registered medical facility using the transmitted location data and forwarded the crash details to the respective hospital.
- Multiple tests confirmed the system's reliability in detecting crashes and accurately sending location and rider details within a minimal delay, ensuring timely assistance.

Objective 2: Trigger a beeping sound upon detecting a crash

- Status: Fully Achieved
- Validation:
 - A buzzer module was integrated into the helmet module and activated when a crash was detected by the sensors.
 - Simulated crash scenarios were tested, and the buzzer reliably emitted an audible alert, notifying nearby individuals of the incident.
 - The beeping functionality worked consistently across various testing environments, including indoor and outdoor setups.

Objective 3: Develop a Web Interface to Provide Rider Information and Crash Location

- Status: Fully Achieved
- Validation:
 - A web interface was designed using the MERN stack, providing real-time rider information and crash data.
 - The user-friendly dashboard displayed critical information, including the rider's name, contact details, and the crash's GPS location.
 - Medical facilities received crash notifications via their dedicated hospital interface, which included the crash site coordinates and rider details.

- End-to-end testing validated the seamless flow of data from the helmet module to the web interface, ensuring accurate and timely display of information.
- Usability tests confirmed that the interface was intuitive and responsive, enabling quick access to necessary data for emergency responders.

CONCLUSION & FUTURE SCOPE

6.1 Conclusions

In conclusion, the IoT-based Smart Helmet project has made significant strides in both the software and hardware domains, aligning with its goal of enhancing rider safety through technology. One of the core achievements has been the successful development of a comprehensive web interface that serves as the central hub for monitoring crash detection using the helmet module. The interface, supported by MongoDB, allows for real-time data visualization and secure storage. The focus on user-centric design and secure data management ensures that the system remains both functional and accessible to users like rider, hospitals, and administrators.

On the hardware side, the integration of essential IoT components has been a major accomplishment. Critical sensor such as the MPU6050 accelerometer-gyroscope module have been successfully incorporated, enabling precise motion detection and location tracking(mobile in which rider interface is logged in). These components along with a buzzer for alert notifications, are all managed by the NodeMCU microcontroller. The seamless communication between these modules and the processing of real-time sensor data has paved the way for implementing key features like crash detection and emergency alerts. The project's robust data communication and processing framework ensures that the helmet can respond promptly to emergencies while providing accurate crash data, marking a significant step toward developing a reliable and intelligent safety solution for riders.

6.2 Environmental Benefits

The IoT Smart Helmet project offers significant environmental benefits by promoting safer and more efficient transportation practices.

Timely detection of accidents ensures quicker medical assistance, reducing the time emergency vehicles spend idling or navigating traffic. This not only minimizes fuel usage but also decreases the carbon footprint associated with prolonged response times.

Additionally, by mitigating the severity of accidents through prompt alerts, the helmet contributes to reducing the environmental impact of repairing or replacing heavily damaged vehicles, as fewer resources are required for extensive repairs or new manufacturing.

Encouraging safer riding practices through real-time alerts also fosters more sustainable motorcycle use, potentially increasing vehicle lifespan. This reduces the environmental burden associated with frequent manufacturing and disposal of vehicle components, contributing to a more sustainable approach to transportation.

These combined benefits highlight the project's potential to enhance road safety while positively impacting the environment.

6.3 Reflections

The IoT-based Smart Helmet project has achieved significant milestones, aligning closely with its core objectives. The following accomplishments outline the progress made so far:

1. Development of Web Interface and Backend Integration: A primary goal of the smart helmet project is to create a platform for providing crash-related information to users and medical facilities. In line with this, we developed a web interface with a user-friendly login page and a dashboard. The backend is powered by MongoDB, ensuring secure storage and retrieval of crash data and rider details. The dashboard is designed to display rider information and the crash location in real time, facilitating efficient communication with nearby medical facilities.

2. Integration and Testing of IoT Components: The core functionality of the smart helmet relies on its integration of advanced sensors and modules with the NodeMCU microcontroller. We successfully incorporated the MPU6050 accelerometer-gyroscope sensor for crash detection, a buzzer module to trigger sound alerts upon crash detection, and accurate crash location details using the mobile to which the helmet is connected. Rigorous testing was conducted to ensure seamless operation of these components, both individually and in communication with the NodeMCU.

3. Data Communication and Processing: We established robust communication channels to ensure real-time transmission of crash-related data from the sensors to the NodeMCU. This data is then processed and transmitted to the backend, where it is used to automatically notify nearby medical facilities with precise coordinates and rider details. This integration is critical to achieving the helmet's emergency response functionality, ensuring timely assistance in case of accidents.

6.4 Future work

The future work of the IoT-based Smart Helmet project will focus on refining and implementing the remaining core features that align with the approved objectives:

Automated Crash Detection and Emergency Response: The next step involves integrating and fine-tuning the automated crash detection system. By leveraging data from the MPU6050 sensor, we will detect collisions and immediately trigger an alert that sends precise GPS coordinates along with rider details to nearby medical facilities. This feature is essential for ensuring timely assistance during accidents, directly supporting rider safety.

Crash Alert Mechanism: We will implement a crash detection system that triggers a buzzer alert if the rider exceeds a predefined threshold limit. The system will be calibrated to recognize jerk thresholds and activate a beeping sound to warn the rider when the limit is crossed, thereby aligning with our objective of enhancing safety.

Trip Data Logging and Parental Control Dashboard: The final focus area will involve the completion of the GPS-based trip data logging and its integration with the web dashboard. The recorded trip data will be accessible through a user-friendly interface, allowing parents or guardians to monitor helmet usage and rider behavior.

7.1 Challenges Faced

Hardware Integration and Sensor Calibration

1. Integrating multiple hardware components like the accelerometer, gyroscope, NodeMCU, and sensors, while resolving wiring errors and power supply issues.
2. Several components were damaged during development and had to be replaced, increasing project costs and time.
3. Ensuring sensor accuracy through extensive calibration to differentiate crashes from normal road vibrations (e.g., speed breaker, potholes etc.).

Reliable Communication and Data Transmission

1. Maintaining seamless transmission of emergency signals and real-time GPS data (from smartphone) under poor network conditions.
2. Overcoming challenges like latency, data loss, and connectivity issues in low-coverage areas to ensure uninterrupted alerts.

Backend Development and Data Management

1. Building a robust backend to handle large data volumes, such as GPS coordinates (from smartphone) and user information.
2. Ensuring data integrity, fast retrieval, and security against unauthorized access.

Comprehensive Testing and Scalability

1. Conducting extensive testing to address edge cases like varying crash intensities, road conditions, and network failures.
2. Balancing affordability and scalability to make the helmet accessible to a broader user base without compromising quality or performance.

Inaccuracy in Displaying user's Current Location after Crash Detection

1. The system may face challenges in accurately pinpointing the user's location during or after a crash due to GPS signal (from smartphone) issues, especially in areas with poor satellite visibility (e.g., under tunnels, dense foliage, or urban canyons).

2. Delays in processing GPS data (from smartphone) or transmitting it to the server can further hinder real-time emergency response, requiring optimization of location-tracking algorithms and backup mechanisms like cellular triangulation.

Power Optimization

1. Managing the power demands of components like GPS (from smartphone), sensors, and communication modules to ensure sufficient battery life.
2. Implementing energy-efficient hardware and power-saving modes without compromising performance.

7.2 Relevant Subjects

Relevant Subjects for the IoT-Based Smart Helmet Project

1. Software Engineering: Essential for designing, developing, and maintaining the software system, including the integration of microcontrollers, helmet-to-bicycle communication modules, and cloud-based emergency response mechanisms.
2. Internet of Things (IoT): Core for integrating hardware components such as accelerometer, gyroscope, NodeMCU to collect, process, and transmit real-time data for accident detection and alerts.
3. Full-Stack Website Development with Next.js: Utilized for building a responsive and dynamic user interface for real-time monitoring dashboards and data visualization. Backend development with Next.js and MongoDB supports server-side rendering for fast data processing and efficient database management of user and vehicle data.
4. Database Management Systems (DBMS): MongoDB is used as a NoSQL database for storing accident logs, GPS coordinates (from smartphone), hospital records, and user details, ensuring secure and efficient data handling with scalability for real-world usage.
5. Data Structures and Algorithms (DSA): Crucial for implementing optimized algorithms for crash detection, geo-location updates, and routing emergency signals efficiently with minimal latency.
6. Real-Time Systems: Ensures time-sensitive operations like accident detection and emergency response are executed with precision, minimizing delays in alerts and updates.

7. **Programming and User Interface Development:** The system is programmed using Arduino IDE, while the user interface is designed with Next.js for web-based dashboards for real-time mobile monitoring and control.

This combination of subjects and tools ensures a robust and seamless integration of hardware and software, enabling the successful implementation of the IoT-Based Smart Helmet system.

7.3 Interdisciplinary Knowledge Sharing

The IoT-Based Smart Helmet project integrated expertise from multiple disciplines to address complex challenges and deliver a comprehensive solution:

1. **Internet of Things (IoT):** Team members with IoT expertise programmed Arduino Uno and integrated sensors (accelerometers, gyroscopes) to collect and process real-time data for accident detection and emergency alerts.
2. **Software Engineering and Web Development:** Developers proficient in Software Engineering and Full-Stack Development built the system's cloud-based backend, APIs, and user interface using Next.js and MongoDB. They ensured seamless communication between hardware components and the cloud while maintaining efficient database management.
3. **Wireless Communication Technologies:** Worked on integrating NodeMCU to enable reliable data transmission for helmet-to-site communication and emergency notifications.
4. **Data Analysis and Real-Time Systems:** Algorithms for crash detection and geo-location were optimized by team members with expertise in Data Structures and Algorithms, ensuring fast and accurate emergency response.
5. **Cloud Computing and Data Management:** Team members with cloud computing knowledge facilitated scalable data storage, real-time processing, and integration with edge servers to relay critical data to hospitals and emergency contacts.

This interdisciplinary collaboration bridged theoretical concepts with practical implementation, enabling the creation of a robust, scalable IoT-Based Smart Helmet system designed to enhance road safety and emergency response.

7.4 Peer Assessment Matrix

Table 13 is a matrix of the contribution of each team member on a scale of 1 (minimum) to 5 (maximum):

Table 9: Peer Assessment Matrix

		Evaluation of:			
		Lovish	Soham	Garv	Aryan
Evaluation by:	Lovish		5	5	5
	Soham	5		5	5
	Garv	5	5		5
	Aryan	5	5	5	

7.5 Role Playing and Work Schedule

Table 10: Role Played by each team member

S. No.	Name	Responsibility
1.	Lovish Bansal	Arduino Coding, DBMS, Backend, Project Integration.
2.	Soham Kumar	Project Testing, UI/UX, Back-end Architecture, Front-end Architecture, Project Documentation.
3.	Garv Bundel	Project Planning, Hardware Selection and Assembly, Arduino Coding, Signal Transmission and communication.
4.	Aryan Tyagi	Website Development (Frontend & Backend), server management, Project Documentation.

TASKS	FEBRUARY	MARCH	APRIL	MAY	JUNE	JULY	AUGUST	SEPTEMBER	OCTOBER	NOVEMBER	DECEMBER
Identification, Formulation and Planning of the Project											
Study of Sensors and Devices											
Hardware assembly and arduino programming											
Database creation and integration											
Signal Transmission and signal testing											
Modifications (if any)											
Website Development											
Data processing and frontend work											
Testing and Optimisations											
Results Evaluation											
Final Report											

Figure 22: Work Schedule

7.6 Student Outcomes Description and Performance Indicators (A-K Mapping)

Table 11: A-K Mapping

Outcome	Performance Indicator	Application	Rating (1-5)
A	Ability to apply knowledge of math, science, and engineering principles.	Section: Chapter 5.3.2 - Algorithmic Approaches Illustrates how we find the nearest hospital in the event of a crash.	5
B	Ability to design and conduct experiments, as well as analyze and interpret data.	Section: Chapter 5.1 - Experimental Setup Describes the deployment architecture.	4
C	Ability to design a system, component, or process to meet desired needs.	Section: Chapter 4.1.2 - System Architecture Outlines the MVC-based architecture for the Crash Detection System.	4

D	Ability to function effectively on a multidisciplinary team.	Section: <i>Chapter 7.3 - Interdisciplinary Knowledge Sharing</i> Highlights collaboration among team roles.	5
E	Ability to identify, formulate, and solve engineering problems.	Section: <i>Chapter 1.1.2 - Problem Statement</i> Defines the challenges and goals of the crash detection system.	5
F	Understanding of professional and ethical responsibility.	Section: <i>Chapter 2.2.4.3 - Security Requirements</i> Ensures user data privacy and compliance with relevant standards.	5
G	Effective communication skills.	Section: <i>Chapter 3.3 - Work Breakdown Structure</i> Demonstrates organized task management and clear communication.	5
H	Broad education necessary to understand the impact of solutions in a global, economic, environmental, and societal context.	Section: <i>Chapter 6.2 - Environmental, Economic, and Societal Benefits</i> Explores the system's impact on road safety.	5
I	Recognition of the need for, and an ability to engage in, lifelong learning.	Section: <i>Chapter 6.4 - Future Work</i> Identifies the importance of ongoing learning for system scalability.	5
J	Knowledge of contemporary issues.	Section: <i>Chapter 1.3 - Research Gaps</i> Discusses the current limitations and opportunities in crash detection.	4
K	Ability to use the techniques, skills, and modern engineering tools necessary for engineering practice.	Section: <i>Chapter 3.4 - Tools and Technology Used</i> Details the implementation of the crash detection system using various tools and techniques.	5

7.7 Brief Analytical Assessment

Our team followed a structured and iterative approach to design, develop, and optimize the IoT-Based Smart Helmet system. We extensively reviewed relevant research papers, technical documentation, and articles related to IoT systems, crash detection, real-time communication, and web-based monitoring platforms to build a robust solution.

We utilized publicly available datasets and real-time testing scenarios to simulate accidents and assess the system's accuracy. By referring to comprehensive documentation for the tools and frameworks used—such as Next.js, MongoDB, Arduino IDE, we ensured smooth development and efficient integration of various components. Feedback from our mentors and assessors at different stages further refined our approach.

The methodologies employed in the project include:

1. **Analytical Methodology:** Conducted a thorough analysis of existing IoT-based safety systems and discussed their practical applications, advantages, and limitations. Researched sensor technologies, data communication protocols, and crash detection algorithms to identify the best-fit solutions for our system.
2. **Computational Methodology:** Designed and implemented crash detection algorithms using accelerometer and gyroscope data. Developed server-side logic with Next.js and MongoDB to ensure efficient data storage and retrieval for real-time operations.
3. **Experimental Methodology:** Conducted extensive testing in diverse scenarios, including different crash intensities, road conditions, and network environments, to validate system performance. Fine-tuned sensor sensitivity and optimized power consumption for consistent and accurate results.

Our capstone project demonstrated a strong foundation in software engineering principles, including modular design, agile development, and system integration. Key components of our implementation included:

Real-time data processing using IoT modules.

Full-stack development with Next.js for the web dashboard and MongoDB for backend database management.

Programming microcontrollers (Arduino Uno and NodeMCU) for sensor data collection and wireless communication.

We adopted the agile development framework to facilitate collaboration, with regular meetings, task , and progress updates. Platforms like GitHub, Google Meet helped task management.

To expand our technical knowledge beyond classroom learning, we utilized resources such as online tutorials, documentation from Arduino and Next.js, and forums like Stack Overflow for debugging and best practices. This capstone project not only enhanced our technical skills in IoT, web development, and system design but also provided valuable insights into teamwork, problem-solving, and addressing real-world challenges with interdisciplinary knowledge.

APPENDIX A: REFERENCES

- [1] Anitha, Anusha, Prema, Rumana Anjum, Akshatha , “Smart Helmet for Safety and Accident Detection using IOT,” International Research Journal of Engineering and Technology (IRJET), vol. 06 Issue, p. 4, 2019.
- [2] V. R. Ch, S. Chegondi, K. Myla, S. Murala, S. S. Myla and D. R. Kalava , “IoT Based Smart Helmet for Bike Rider Safety,” IEEE, no. 10.1109, p. 4, 2023.
- [3] Rahul Kamdi, Rohan Vaidya, Deepali Borakhade, Prasheel Thakre, Ajinkya Nilawar, Vishal Panchbhai , “An IoT Based Intelligent and Smart Helmetfor Bike RidersUsing Arduino,” International Journal of Intelligent Systems and Applications in Engineering, vol. 12, no. 2147-6799, p. 6, 2024.
- [4] M. E. Alim, S. Ahmad, M. N. Dorabati and I. Hassoun , “Design & Implementation of IoT Based Smart Helmet for Road Accident Detection,” IEEE, vol. 11, no. 2644 3163, p. 5, 2020.
- [5] Monagi H. Alkinani , Abdulwahab Ali Almazroi, NZ Jhanjhi, and Navid Ali Khan , “5G and IoT Based Reporting and Accident Detection (RAD) System to Deliver First Aid Box Using Unmanned Aerial Vehicle,” MDPI Journals, vol. 21, no. 6905, p. 16, 2021.
- [6] Guntupalli Sireesha, Anusha N, K. Baby Satya Jahnavi, Ayusha Baburay , “Smart Helmet using IoT,” International Journal of Engineering Research & Technology (IJERT), vol. 8, no. 14, p. 5, 2020.
- [7] Md. Atiqur Rahman, S.M Ahsanuzzaman, Ishman Rahman, Toufiq Ahmed, Abid Ahsan , “IoT Based Smart Helmet and Accident Identification System,” IEEE Xplore, vol. IEEE Region 10 Symposium, no. 2642-6102, p. 5, 2020.
- [8] Mangesh Jadhwar, Gauri Kandepalli, Ashlesha Kohade, Rajkumar Komati , “SMART HELMET SAFETY SYSTEM USING ATMEGA 32,” International Journal of Research in Engineering and Technology, vol. 05, no. 05, p. 3, 2016.
- [9] S. S. A. M. M. I. R. P. a. P Prasetyawan1, "A prototype of IoT-based smart system to support," Journal of Physics: Conference Series, no. 10.1088/1742-6596/1810/1/012005,p.10,2021.
- [10] P. Pathak, "IoT based Smart Helmet with Motorbike Unit for," IEEE, no. 10.1109/ICACCCN51052.2020.9362986,p.7,2020.

APPENDIX B: PLAGIARISM REPORT



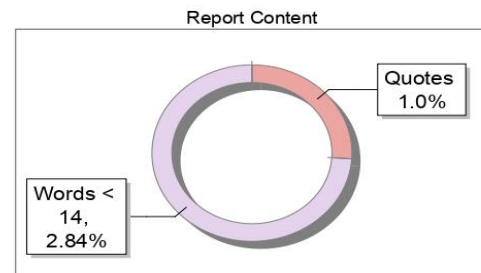
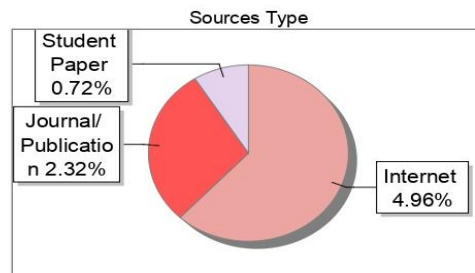
The Report is Generated by DrillBit Plagiarism Detection Software

Submission Information

Author Name	Capstone G49
Title	Smart Helmet
Paper/Submission ID	2804853
Submitted by	vivek.sethi@thapar.edu
Submission Date	2024-12-18 21:17:11
Total Pages, Total Words	101, 19066
Document type	Others

Result Information

Similarity **8 %**



Exclude Information

Quotes	Not Excluded
References/Bibliography	Not Excluded
Source: Excluded < 14 Words	Not Excluded
Excluded Source	0 %
Excluded Phrases	Not Excluded

Database Selection

Language	English
Student Papers	Yes
Journals & publishers	Yes
Internet or Web	Yes
Institution Repository	Yes

A Unique QR Code use to View/Download/Share Pdf File

